

On the construction of quantum circuits for S-boxes with different criteria based on the SAT solver

Da Lin¹, Chunli Yang², Shengyuan Xu³, Shizhu Tian²
and Bing Sun^{1*}

¹College of Science, National University of Defense Technology,
Changsha, 410073, China.

²School of Cyber Science and Technology, Hubei University,
Wuhan, 430062, China.

³Department of Fundamental Courses, Shandong University of
Science and Technology, Taian, 271019, China.

*Corresponding author(s). E-mail(s): happy_come@163.com;
linda_is@163.com; yyli@stu.hubu.edu.cn;
xushengyuan@sdust.edu.cn; Shizhutian@hubu.edu.cn;

Abstract

The substitution box (S-box) is often used as the only nonlinear component in symmetric-key ciphers, leading to a significant impact on the implementation performance of ciphers in both classical and quantum application scenarios by S-box circuits. Taking the Pauli-X gate, the CNOT gate, and the Toffoli gate (i.e., the NCT gate set) as the underlying logic gates, this work investigates the quantum circuit implementation of S-boxes based on the SAT solver. Firstly, we propose encoding methods of the logic gates and the NCT-based circuit, based on which we construct STP models for implementing S-boxes. By applying the proposed models to the S-boxes of several well-known cryptographic algorithms, we construct optimal implementations with different criteria for the 4-bit S-boxes and provide the implementation bounds of different criteria for the 5-bit S-boxes. Since S-boxes in the same affine equivalence class share most of the important properties, we then build STP models to further investigate optimizing S-box circuits based on affine equivalence. According to the applications, for almost all the tested 4-bit S-boxes, there always exists an equivalent S-box that can

be implemented with half the number of logic gates. Besides, we encode some important cryptographic properties and construct STP models to design S-boxes with given criteria configurations on implementation and properties. As an application, we find an S-box with the same cryptographic properties as the S-box of KECCAK that can be implemented with only 5 NCT gates, even though the application of our models indicates that implementing the KECCAK S-box requires more than 9 NCT gates. Notably, the inputs of the proposed models are tweakable, which makes the models possess some functions not currently available in the public tools for constructing optimized NCT-based circuits for S-boxes.

Keywords: S-box, implementation, the NCT gate set, the SAT solver

1 Introduction

As an important part of modern cryptography, the symmetric-key cryptographic primitives is a key technique to ensure the confidentiality and integrity of information, and thus its security is crucial. When designing a symmetric-key cryptographic primitive, it is necessary to ensure that the algorithm has the ability to resist known cryptographic attacks, such as differential cryptanalysis [1] and linear cryptanalysis [2]. With the rapid development of quantum technology, the symmetric-key cryptographic primitive also has to face the threat of quantum attacks, as the parallelism of quantum computing makes quantum computers possess advantages over classical ones in dealing with some specific problems [3–5].

Considering the threat posed in the forthcoming post-quantum era, the optimized quantum implementation of the symmetric-key cryptographic primitive has become an active research topic in recent years, such as [6–9]. The research interest of the cryptographic community in this field is mainly aroused by the fact that the quantum implementation of a cipher is an important component of the circuit for evaluating its quantum security level [10, 11] or conducting quantum attacks against it. In addition, considering the limited scale of the current quantum computers, it is widely believed that the optimized quantum implementation circuits of cryptographic algorithms can be instantiated earlier in the future when quantum computers become robust enough.

For symmetric-key cryptographic primitives, *confusion* and *diffusion* [12] are two important cryptographic criteria, the establishment of which is primarily dependent on the nonlinear layer and the linear layer respectively. Generally, the Substitution box (S-box) is often used as the only nonlinear component of the symmetric-key cryptographic primitive, and thus its implementation and properties have great impacts on that of the whole cipher.

To construct optimized implementation schemes for S-boxes, there are many researches focusing on the development of tools. In 2017, Jean *et al.* [13]

introduced **LIGHTER**, a tool that aims at implementing small S-boxes with small area in hardware application scenarios or fewer instructions in software application scenarios. To this end, several standard cell libraries are encoded in the tool, as well as the corresponding cost metric of each instruction. Later, the platform **PEIGEN** was presented at ToSC 2019 in [14]. In addition to constructing efficient hardware/software implementations of S-boxes, **PEIGEN** can also be applied to evaluate security and generate S-boxes. Notably, the instructions encoded in **LIGHTER** are consistent with the *in-place* property¹ of quantum logic gates. As a consequence, the reversible implementation returned by **LIGHTER** can also be applied to construct quantum circuits, especially the ones built with the NCT gate set [15–17]. Following the line of the research on constructing optimized implementations for S-boxes, **LIGHTER-R** [18] and **DORCIS** [19] are built based on **LIGHTER** for quantum application scenarios. The former constructs quantum circuits at the top level with the NCT gate set, while the latter considers the decomposition of the Toffoli gate and constructs Clifford+ T -based circuits.

Most of the above-mentioned tools are based on heuristic algorithms. Although they can return good solutions, but it is relatively hard to prove those solutions are optimal. There are researches on searching for the optimal classical circuit implementations of S-boxes. In [20], Stoffelen defined a set of equations to represent the Algebraic Normal Form (ANF) of the S-box and converted them into Conjunctive Normal Form (CNF). By applying the SAT solver, the author constructed optimal implementations for several small S-boxes under various criteria. Later, Bilgin *et al.* [21] proposed methods to optimize the AND-depth, and Fan *et al.* [22] introduced 3-input gates to optimize the area. Recently, Feng *et al.* [23] proposed two methods to encode the implementations of S-boxes and introduced new SAT-based search methods.

In addition to the construction of optimized implementation schemes, the SAT solver has also been used to design new S-boxes [24]. In this case, some important cryptographic properties of S-boxes should be considered, such as differential uniformity, linearity, and their frequency. Bijective S-boxes with lowest differential uniformity and linearity are considered optimal, as they help to resist differential and linear cryptanalysis respectively. Besides, the frequency of differential uniformity in the differential distribution table (DDT) and that of linearity in the linear approximation table (LAT) are also considered to have important impacts on the attacks [24, 25]. Additionally, the number of active S-boxes is closely related to the probability of differential and linear attacks, a small number of active S-boxes will increase the probability of a successful attack. Therefore, the input and output differentials/masks of S-boxes with a Hamming weight of 1 (named bad input and bad output pattern, i.e., BIBO pattern, in [24]) would serve to perform differential/linear attacks.

This work investigates the optimization of S-box implementations constructed based on the NCT gate set by applying the SAT solver. It should

¹The *in-place* property requires that no additional variables (qubits in quantum setting) are introduced for keeping the output of any operations.

be noted that the researches in [26] reveal that at least one additional ancilla qubit is required for implementing an odd permutation with NCT gates. In this work, we only focus on constructing optimized NCT-based circuits for even permutations for saving qubits.

Firstly, we encode the NCT gate set and the circuits built based on those gates, based on which we construct STP models to identify the optimal implementations of S-boxes for different criteria. As an application, we apply the proposed models to the 4-/5-bit S-boxes adopted in several well-known cryptographic algorithms. The results reveal that for those 4-bit S-boxes, the optimal implementations for all tested criteria can be constructed. For the 5-bit S-boxes, although the SAT solver cannot return their optimal implementation schemes in a reasonable time, the bounds of different criteria can be provided.

Moreover, we further investigate optimizing S-box implementations based on affine equivalence. The results show that the optimal implementation of a given 4-bit S-box is not necessarily optimal among implementations of all the S-boxes in the same affine equivalence class. For most tested 4-bit S-boxes, there is always an S-box with/without fixed points and affine equivalent to the tested one but can be implemented with only about half of the gate count. For the 4-bit S-boxes with/without fixed points and affine equivalent to the optimal ones corresponding to even permutations, at least 4/5 gates are required to build their NCT-based circuits.

Additionally, by combing the encoding methods of the NCT-based circuits and the S-box properties, we construct models to search for S-boxes with given constraints on the implementation and various cryptographic properties. In our experiments, the SAT solver returns 3456 optimal 4-bit S-boxes with fixed points. Further observations show that only 4 NCT gates are enough to implement the S-boxes. Besides, each of those S-boxes contains the fixed point $S[0] = 0$. By placing restrictions on the Toffoli gate consumption, it can be verified that at least 4 Toffoli gates are required to construct NCT-based circuits for the optimal 4-bit S-boxes, which also implies that those S-boxes can be implemented at least in a Toffoli-depth of 4 without introducing additional ancilla qubits.

The rest of this paper is organized as follows: In Section 2, notations used throughout the paper, the SAT solver, some cryptographic properties of the S-box, and the NCT gate set are introduced. Section 3 provides the models of encoding the NCT gates and NCT-based circuits, as well as that for optimizing S-box circuits. The investigation of optimizing the NCT-based circuits of the S-box based on its properties is given in Section 4. Section 5 presents the application and Section 6 concludes the work.

2 Preliminaries

2.1 Notations

Notations used throughout this paper are presented in Table 1.

Table 1 Notations.

Notation	Description
$\mathbb{F}_2$	The finite field with two elements 0 and 1
$\mathbb{F}_2^n$	The n -dimensional vector space over $\mathbb{F}_2$
$a \oplus b$	The XOR of bits a and b over $\mathbb{F}_2$
$a \cdot b$	The AND of bits a and b over $\mathbb{F}_2$
$wt(a)$	The Hamming weight of a
$a b$	The concatenation of a and b
$\wedge$	Logical operation AND
$\vee$	Logical operation OR

2.2 The SAT Problem and Its Extension

The Boolean Satisfiability (SAT) problem studies whether there is a set of Boolean variables whose assignment satisfies the given Boolean formulas. If so, the problem is said to be satisfiable; otherwise, it is said to be unsatisfiable. The SAT problem is a deterministic problem and has been proven to be Non-deterministic Polynomial Complete (NPC).

Usually, for a given Boolean formula, researchers first represent it as the standard input format of SAT solver, namely the form of CNF $\bigwedge_{i=0}^{m-1} \left(\bigvee_{j=0}^{n_i-1} x_{i,j} \right) = 1$, where $x_{i,j}$ are variables, constants or the negation of Boolean variables. There are also many mature and efficient solvers for SAT problems, such as Minisat [27], Cryptominisat [28], and CaDiCal [29].

The Satisfiability Modulo Theories (SMT) problem is an extension of the SAT problem, in which certain Boolean variables are substituted with a suitable set of binary and (or) non-binary variables. STP [30] is a constraint solver for the theory of quantifier-free bitvectors that can solve the SMT problem, which is mainly used in previous researches. For STP, the CVC format is one of the frequently employed file-based input languages. For additional information, please refer to <http://stp.github.io/>.

2.3 Cryptographic Properties of the S-box

When designing an S-box, the cryptographic properties of the S-box should be extensively analyzed, as they can propagate along with the round function and may eventually affect the security of the cipher. Differential cryptanalysis and linear cryptanalysis are the two most well-known attack methods. To judge the ability of cryptographic algorithms to resist those attacks, special attention has been paid to differential uniformity and linearity of the S-box.

The differential uniformity of an S-box is defined by its DDT.

Definition 1 (Differential Distribution Table, Differential Uniformity [31]) Let $S : \mathbb{F}_2^n \rightarrow \mathbb{F}_2^m$ be a vectorial Boolean function. For any $a \in \mathbb{F}_2^n, b \in \mathbb{F}_2^m$, define

$$\delta_S(a, b) = \#\{x \in \mathbb{F}_2^n | S(x) \oplus S(x \oplus a) = b\}. \quad (1)$$

The differential distribution table of S is a 2-dimensional table with size $2^n \times 2^m$, in which the element $\text{DDT}[a, b]$ in row a and column b is equal to $\delta_S(a, b)$, where $a \in \mathbb{F}_2^n, b \in \mathbb{F}_2^m$. The differential uniformity of S is defined as:

$$\mathcal{U}(S) \triangleq \max_{a \in \mathbb{F}_2^n \setminus \{0\}, b \in \mathbb{F}_2^m} \delta_S(a, b). \quad (2)$$

Similarly, the linearity of an S-box is defined by its LAT.

Definition 2 (Linear Approximation Table, Linearity of an S-box [32]) Let $S : \mathbb{F}_2^n \rightarrow \mathbb{F}_2^m$ be a vectorial Boolean function. For any $a \in \mathbb{F}_2^n, b \in \mathbb{F}_2^m$, define

$$\lambda_S(a, b) = \#\{x \in \mathbb{F}_2^n \mid a \cdot x \oplus b \cdot S(x) = 0\} = \frac{1}{2} \sum_{x \in \mathbb{F}_2^n} (-1)^{a \cdot x \oplus b \cdot S(x)}. \quad (3)$$

The linear approximation table of S is a $2^n \times 2^m$ table with the element $\text{LAT}[a, b]$ in row a and column b equals $\lambda_S(a, b)$, where $a \in \mathbb{F}_2^n, b \in \mathbb{F}_2^m$. The linearity of S is defined as:

$$L(S) \triangleq \max_{a \in \mathbb{F}_2^n, b \in \mathbb{F}_2^m \setminus \{0\}} |2 \cdot \lambda_S(a, b)|. \quad (4)$$

Notably, for the S-boxes in the same affine equivalence class, the differential uniformity and the linearity can be preserved.

Definition 3 (Affine Equivalence) Two S-boxes S_1 and S_2 are affine equivalent, if there exist two reversible linear transformations $A, B \in GL(n, \mathbb{F}_2)$ and two constants $a, b \in \mathbb{F}_2^n$ such that

$$S_2 = B(S_1(A(x) \oplus a)) \oplus b. \quad (5)$$

In addition, the frequency of the differential uniformity in DDT and that of the linearity in LAT, as well as BIBO patterns derived from DDT and LAT have also received much attention, as they may affect the probability of a successful attack.

Definition 4 (Frequency of Differential Uniformity) For the DDT of a given vectorial Boolean function $S : \mathbb{F}_2^n \rightarrow \mathbb{F}_2^m$, the frequency of differential uniformity in DDT is defined as:

$$\mathcal{U}_{Freq} \triangleq \#\{(a, b) \mid \delta_S(a, b) = \mathcal{U}(S), a \in \mathbb{F}_2^n \setminus \{0\}, b \in \mathbb{F}_2^m\}. \quad (6)$$

Definition 5 (Frequency of Linearity) For the LAT of a given vectorial Boolean function $S : \mathbb{F}_2^n \rightarrow \mathbb{F}_2^m$, the frequency of linearity in LAT is defined as:

$$\mathcal{L}_{Freq} \triangleq \#\{(a, b) \mid |2 \cdot \lambda_S(a, b)| = \mathcal{L}(S), a \in \mathbb{F}_2^n, b \in \mathbb{F}_2^m \setminus \{0\}\}. \quad (7)$$

Definition 6 (Bad Input and Bad Output Pattern [24]) Given the DDT/LAT of a vectorial Boolean function $S : \mathbb{F}_2^n \rightarrow \mathbb{F}_2^m$, if there exists a pair $(a, b) \in \mathbb{F}_2^n \times \mathbb{F}_2^m$ satisfying that $\text{DDT}[a, b] \neq 0 / \text{LAT}[a, b] \neq 0$ and $wt(a) = wt(b) = 1$. Then, (a, b)

is called a bad input and bad output (BIBO) pattern in DDT/LAT. Denote by $\#BIBO_{DDT}$ and $\#BIBO_{LAT}$ the number of BIBO patterns in the DDT and LAT of S . Then, $\#BIBO_{DDT}$ and $\#BIBO_{LAT}$ can be calculated as:

$$\begin{cases} \#BIBO_{DDT} = \#\{(a, b) | DDT[a, b] \neq 0, wt(a) = wt(b) = 1, a \in \mathbb{F}_2^n, b \in \mathbb{F}_2^m\}, \\ \#BIBO_{LAT} = \#\{(a, b) | LAT[a, b] \neq 0, wt(a) = wt(b) = 1, a \in \mathbb{F}_2^n, b \in \mathbb{F}_2^m\}. \end{cases} \quad (8)$$

2.4 The NCT-based Circuit and Its Criteria

The NCT-based circuit is constructed based on the NCT gate set.

The NCT Gate Set The NCT gate set consists of the Pauli-X gate, the CNOT gate and the Toffoli gate, the circuit symbols of which are depicted in Figure 1, where $a, b, c \in \mathbb{F}_2$.

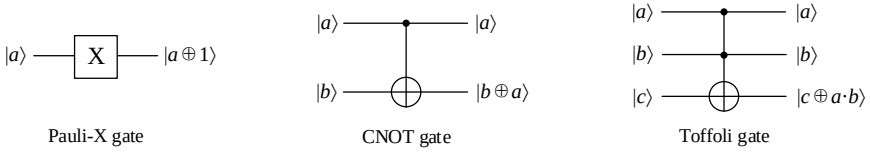


Fig. 1 The description of the NCT gates.

- The Pauli-X gate (or the NOT gate) is a single-qubit gate, and reverses the state of the qubit, i.e., transforms $|a\rangle$ to $|a \oplus 1\rangle$.
- The CNOT gate, also known as the Controlled-NOT gate or the C-NOT gate, reverses the state of the target qubit if the state of the control qubit is $|1\rangle$. The function of the CNOT gate can be represented as the mapping that transforms $|a\rangle|b\rangle$ to $|a\rangle|b \oplus a\rangle$.
- The Toffoli gate, or the CC-NOT gate, reverses the state of the target qubit when both the control qubits are with state $|1\rangle$. The function of the Toffoli gate can be represented as the mapping that transforms $|a\rangle|b\rangle|c\rangle$ to $|a\rangle|b\rangle|c \oplus a \cdot b\rangle$.

For simplicity, we omit the Dirac notation of the qubits and simply express the functions of the Pauli-X gate, the CNOT gate and the Toffoli gate as $a = a \oplus 1$, $b = b \oplus a$, and $c = c \oplus a \cdot b$ respectively.

Implementation Criteria Efficient implementations of S-boxes are of much importance, as they largely determine the implementation performance of the whole algorithm. Consequently, various criteria are considered when designing optimized implementation schemes for S-boxes. For example, to minimize the latency of the hardware implementation, depth is taken as the metric. For quantum applications, when building LIGHTER-R [18], gate count, two-qubit cost, and quantum cost are taken as the implementation criteria of NCT-based circuits. Actually, evaluating those criteria is equivalent to counting the number of different logic gates for implementing a specific function. Roughly, the criterion gate count considers the number of the NCT gates, the two-qubit cost is defined based on the number of 2-input gates, the quantum cost

counts the number of Pauli-X, CNOT, and Controlled-V/-V[†] gates (see [33] for details). According to [18, 33], the costs of NCT gates under different criteria are listed in Table 2.

Table 2 Cost metrics of the NCT gates

gate criterion	the Pauli-X gate	the CNOT gate	the Toffoli gate
gate count	1	1	1
two-qubit cost	0	1	5
quantum cost	1	1	5

In the NCT gate set, the implementation cost of the Toffoli gate is much more expensive than that of other gates. Therefore, the Toffoli gate consumption and the Toffoli-Depth of the NCT-based circuit have also attracted a lot of attention. For an NCT-based circuit, its Toffoli gate consumption counts the number of Toffoli gates required for implementing the circuit, while its Toffoli-Depth equals the layer of Toffoli gates in the circuit that cannot be applied in parallel.

3 Optimizing the S-box Implementation

In this section, the encoding equations related to the input and output of the underlying logic gates are introduced, based on which the construction of the STP models for searching for optimized S-box implementation schemes with various criteria is presented.

3.1 New Encoding Method

The methods of encoding quantum gates are quite different from those of classical ones, as quantum gates are reversible. Besides, when encoding quantum gates, the various number of gate inputs should also be considered. For instance, in [20], S-box implementations are restricted to the gates with input number less than 2 for optimizing various criteria, such as the NOT gate, the AND gate, and the OR gate. However, in the NCT gate set, the Toffoli gate is a 3-input gate. Additionally, for multi-input quantum gates, especially the CNOT gate and the Toffoli gate in the NCT gate set, the inputs are divided into the control qubit and the target qubit, which makes more cases should be considered when encoding the NCT gates. To further explain the logic gates that may be involved in each operation, the following example is given.

Example 1 Given a 4-bit S-box, the input of which is denoted by (x_0, x_1, x_2, x_3) . In each operation, the corresponding NCT gate may change any of the S-box inputs. Suppose that the change occurs in x_0 , i.e., x_0 is applied as the target qubit, the possible quantum gate corresponding to the current operation may act as the following three different classes:

- The Pauli-X gate: As a single-qubit gate, the Pauli-X gate can only overwrite the state of x_0 as $x_0 = x_0 \oplus 1$, as the first gate shown in Figure 2.
- The CNOT gate: In this case, the state of x_0 can be overwritten as $x_0 = x_0 \oplus x_1$, $x_0 = x_0 \oplus x_2$ or $x_0 = x_0 \oplus x_3$, as shown with the 2nd, 3rd and 4th gates in Figure 2.
- The Toffoli gate: The state of x_0 can be overwritten by the Toffoli gate as $x_0 = x_0 \oplus x_1 \cdot x_2$, $x_0 = x_0 \oplus x_1 \cdot x_3$ or $x_0 = x_0 \oplus x_2 \cdot x_3$, as shown in Figure 2 with the last 3 gates.

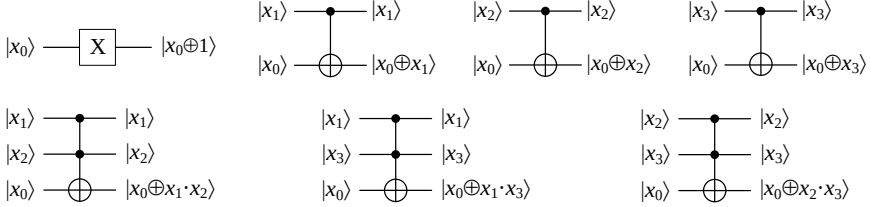


Fig. 2 Possible gates for the case that $|x_0\rangle$ is used as the target qubit.

From Example 1, when implementing a 4-bit S-box with the NCT gate set, there are 7 specific logic gates can be selected as the candidate of the current operation once the target qubit is fixed. It also means that there are a total of 28 different NCT gates can be chosen by each operation as the candidate. Suppose that k gates are consumed for implementing a 4-bit S-box, the complexity of conducting an exhaustive search for an optimal implementation scheme of the S-box is 28^k . For instance, for the case that $k = 8$, the time complexity is approximately $2^{38.46}$. However, in fact, many S-boxes require more than 8 logic gates, making the calculation even more challenging.

Encoding the NCT Gates Three logic gates with different numbers of inputs are contained in the NCT gate set. Denote by q_0^i, q_1^i, q_2^i the inputs of the i -th gate and b_0^i, b_1^i, b_2^i the variables that determine which NCT gate the i -th gate corresponds to. We fix q_0^i as the target qubit of the i -th gate, that is, the state of q_0^i will be overwritten by the current gate. We also use $+$ to represent $\oplus$ in NCT gates without confusion. To encode the NCT gate set, we define the output of the i -th gate, denoted by t^i , as follows:

$$t^i = q_0^i + b_0^i + b_1^i \cdot q_1^i + b_2^i \cdot q_1^i \cdot q_2^i. \quad (9)$$

To maintain the *in-place* property of the quantum gate, the t^i in Eq. (9) will be assigned to the input determined by q_0^i (the encoding of which will be introduced later). The expression of t^i with different values of b_0^i, b_1^i and b_2^i are listed in Table 3.

Denote by f the vectorial Boolean function with input $(x_0, x_1, \dots, x_{n-1})$. Based on Eq. (9) and Table 3, the encoding of the NCT gates for implementing f can be described as follows:

Table 3 Encoding the NCT gate set

(b_2^i, b_1^i, b_0^i)	t^i	corresponding quantum gate
$(0, 0, 1)$	$q_0^i + 1$	the Pauli-X gate
$(0, 1, 0)$	$q_0^i + q_1^i$	the CNOT gate
$(1, 0, 0)$	$q_0^i + q_1^i \cdot q_2^i$	the Toffoli gate

- From Table 3, if $wt(b_0^i b_1^i b_2^i) = 1$ holds, one of the gates in the NCT gate set can be determined, which gives rise to the following restriction:

$$b_0^i + b_1^i + b_2^i = 1. \quad (10)$$

- Quantum gate is reversible and no ancilla qubits are introduced for the purpose of saving qubits in this work, which means that changes in state of the f input will be caused by each gate. Denote by $(x_0^{i-1}, x_1^{i-1}, \dots, x_{n-1}^{i-1})$ the state of the f input before the application of the i -th gate. It follows that variables q_0^i, q_1^i and q_2^i are selected from $\{x_0^{i-1}, x_1^{i-1}, \dots, x_{n-1}^{i-1}\}$, which can be restricted as follows:

$$\begin{cases} q_l^i = \sum_{j=0}^{n-1} a_{n \cdot l + j}^i \cdot x_j^{i-1}, \\ \sum_{j=0}^{n-1} a_{n \cdot l + j}^i = 1, \end{cases} \quad (11)$$

where $l = 0, 1, 2$.

- The qubit can not be copied. It means that loading a qubit into multiple gates simultaneously is not allowed. As a result, q_0^i, q_1^i and q_2^i should be assigned to different inputs of f , which can be restricted by

$$a_j^i + a_{n+j}^i + a_{2n+j}^i \leq 1, \quad (12)$$

where $j \in [0, n-1]$ and n is the number of inputs of f .

- To eliminate symmetry, we restrict that only q_0^i will be changed in Eq. (9), while q_1^i and q_2^i are not. In other words, the update of q_0^i and its relationship with t^i should be encoded to maintain the *in-place* property of the quantum gate. According to Eq. (11), the variable q_0^i in Eq. (9) is restricted as $q_0^i = \sum_{j=0}^{n-1} a_j^i \cdot x_j^{i-1}$ with $\sum_{j=0}^{n-1} a_j^i = 1$, it follows that for any $j \in [0, n-1]$, if $a_j^i = 0$, the corresponding x_j^{i-1} will not be changed by the i -th gate. However, if $a_j^i = 1$, the corresponding input x_j^{i-1} should be updated to the output of the current gate. The above restriction can be described as follows:

$$x_j^i = a_j^i \cdot t^i + (1 + a_j^i) \cdot x_j^{i-1}, \quad (13)$$

where $j \in [0, n-1]$ and n is the number of inputs of f .

Encoding the Circuit By taking the output of the i -th gate, i.e., $(x_0^i, x_1^i, \dots, x_{n-1}^i)$, as the input of the next gate, multiple gates can be encoded. Denote by $(x_0^k, x_1^k, \dots, x_{n-1}^k)$ the output of the k -th gate. If each bit

in $\{x_0^k, x_1^k, \dots, x_{n-1}^k\}$ equals to a different output bit of the given vectorial Boolean function (denoted by $(y_0, y_1, \dots, y_{n-1})$), then an NCT-based circuit of the vectorial Boolean function is constructed. Totally, there are $n!$ possible permutations map $(x_0^k, x_1^k, \dots, x_{n-1}^k)$ to $(y_0, y_1, \dots, y_{n-1})$. The relationship between $(x_0^k, x_1^k, \dots, x_{n-1}^k)$ and $(y_0, y_1, \dots, y_{n-1})$ can be expressed as

$$y_j = \sum_{i=0}^{n-1} m_{j_i} \cdot x_i^k, \quad (14)$$

where $j \in [0, n-1]$. Notably, for $i, j \in [0, n-1]$, m_{j_i} 's in Eq. (14) form a permutation matrix.

The following 3-bit S-box is given as an example to illustrate the complete procedure of encoding the NCT-based circuit for a given vectorial Boolean function.

Example 2 Denote the input and output of the given 3-bit S-box as (x_0, x_1, x_2) and (y_0, y_1, y_2) respectively. The encoding of the first gate and the corresponding constraints are listed as follows:

To encode the inputs of the current gate, the following constraints are given:

$$\begin{cases} q_0^1 = a_0^1 \cdot x_0 + a_1^1 \cdot x_1 + a_2^1 \cdot x_2, \\ q_1^1 = a_3^1 \cdot x_0 + a_4^1 \cdot x_1 + a_5^1 \cdot x_2, \\ q_2^1 = a_6^1 \cdot x_0 + a_7^1 \cdot x_1 + a_8^1 \cdot x_2, \\ a_0^1 + a_1^1 + a_2^1 = 1, \\ a_3^1 + a_4^1 + a_5^1 = 1, \\ a_6^1 + a_7^1 + a_8^1 = 1, \\ a_0^1 + a_3^1 + a_6^1 \leq 1, \\ a_1^1 + a_4^1 + a_7^1 \leq 1, \\ a_2^1 + a_5^1 + a_8^1 \leq 1. \end{cases} \quad (15)$$

The output of the current gate is encoded as

$$\begin{cases} t^1 = q_0^1 + b_0^1 + b_1^1 \cdot q_1^1 + b_2^1 \cdot q_1^1 \cdot q_2^1, \\ b_0^1 + b_1^1 + b_2^1 = 1. \end{cases} \quad (16)$$

After the current gate, the S-box inputs are updated as

$$\begin{cases} x_0^1 = a_0^1 \cdot t^1 + (1 + a_0^1) \cdot x_0, \\ x_1^1 = a_1^1 \cdot t^1 + (1 + a_1^1) \cdot x_1, \\ x_2^1 = a_2^1 \cdot t^1 + (1 + a_2^1) \cdot x_2. \end{cases} \quad (17)$$

The above steps encode the first gate, after which the S-box input is transformed from (x_0, x_1, x_2) to (x_0^1, x_1^1, x_2^1) and will be regarded as the input of the next gate. After the application of k gates (the encoding of each is similar to that of the first gate shown with Eqs. (15) - (17)), the S-box input is transformed to (x_0^k, x_1^k, x_2^k) , corresponding to the S-box output, i.e., (y_0, y_1, y_2) . Thus, we have

$$\left\{ \begin{array}{l} y_0 = m_{0_0} \cdot x_0^k + m_{0_1} \cdot x_1^k + m_{0_2} \cdot x_2^k, \\ y_1 = m_{1_0} \cdot x_0^k + m_{1_1} \cdot x_1^k + m_{1_2} \cdot x_2^k, \\ y_2 = m_{2_0} \cdot x_0^k + m_{2_1} \cdot x_1^k + m_{2_2} \cdot x_2^k, \\ m_{0_0} + m_{0_1} + m_{0_2} = 1, \\ m_{1_0} + m_{1_1} + m_{1_2} = 1, \\ m_{2_0} + m_{2_1} + m_{2_2} = 1, \\ m_{0_0} + m_{1_0} + m_{2_0} = 1, \\ m_{0_1} + m_{1_1} + m_{2_1} = 1, \\ m_{0_2} + m_{1_2} + m_{2_2} = 1. \end{array} \right. \quad (18)$$

3.2 New STP Models

With the encoding of the underlying logic gates, as a direct application, it is possible to apply the SAT solver to search for an optimal implementation of an S-box with the criterion gate count. To this end, Algorithm 1 is introduced based on Section 3.1. Note that all values in the algorithm are determined by the S-box look-up table.

Algorithm 1 is designed to generate STP models for small S-boxes, then we can apply the SAT solver to solve the models. Note that we only present in Algorithm 1 the ANF representation for simplicity. In practical application, it should be converted into the CVC language, which is the input of the solver.

Converting ANF to CVC The ANF involved in this work is converted into the CVC language as follows:

- The definition of variables: For $a \in \mathbb{F}_2$, $b \in \mathbb{F}_2^n$ and $S : \mathbb{F}_2^n \rightarrow \mathbb{F}_2^n$, a, b and S are defined as

```
a: BITVECTOR(1);
b: BITVECTOR(n);
S: ARRAY BITVECTOR(n) OF BITVECTOR(n);
```

- Value assignment: For $a, b \in \mathbb{F}_2$, the initialization of $a = 0$ and $b = 1$ are defined as

```
ASSERT(a = 0bin0);
ASSERT(b = 0bin1);
```

- The concatenation of variables: For $a \in \mathbb{F}_2^n$, $b \in \mathbb{F}_2^m$ and $c \in \mathbb{F}_2^{m+n}$, $c = a||b$ is defined as

```
ASSERT(c = a@b);
```

- The substitution of the XOR and AND operations: For $a, b, c, d \in \mathbb{F}_2$, the bit-wise XOR and AND operations are defined as

```
ASSERT(c = BVXOR(a, b));
ASSERT(d = a&b);
```

Algorithm 1 Construct NCT-based Circuits for the S-box**Require:** Look-up table of the S-box S , the number of gates K .**Ensure:** STPModel(S, K).

```

1: for  $s = 0$  to  $2^n - 1$  do
2:   for  $i = 0$  to  $n - 1$  do
3:      $x_{s_i}^0 = x_{s_i}$ ; ▷ Initialize the inputs
4:   end for
5: end for
6: for  $s = 0$  to  $2^n - 1$  do
7:   for  $k = 1$  to  $K$  do
8:     for  $l = 0$  to  $2$  do
9:        $q_{s_l}^k = \sum_{i=0}^{n-1} a_{n \cdot l + i}^k \cdot x_{s_i}^k$ ;
10:    end for
11:     $t_s^k = q_{s_0}^k + b_0^k + b_1^k \cdot q_{s_1}^k + b_2^k \cdot q_{s_1}^k \cdot q_{s_2}^k$ ; ▷ The  $k$ -th logic gate
12:    for  $i = 0$  to  $n - 1$  do
13:       $x_{s_i}^k = a_i^k \cdot t_s^k + (1 + a_i^k) \cdot x_{s_i}^{k-1}$ ; ▷ Update the inputs of the S-box
14:    end for
15:  end for
16:  for  $j = 0$  to  $n - 1$  do
17:     $y_{s_j} = \sum_{i=0}^{n-1} m_{j\_i} \cdot x_{s_i}^k$ ; ▷ Encode the S-box outputs
18:  end for
19: end for
20: for  $k = 1$  to  $K$  do
21:    $\sum_{i=0}^{n-1} a_{n \cdot l + i}^k = 1$ ;
22:    $\sum_{l=0}^2 a_{n \cdot l}^k \leq 1$ ;
23:    $b_0^k + b_1^k + b_2^k = 1$ ;
24: end for
25: for  $i = 0$  to  $n - 1$  do ▷ Encode the permutation matrix formed by  $m_{i\_j}$ s'
26:    $\sum_{j=0}^{n-1} m_{i\_j} = 1$ ;
27:    $\sum_{j=0}^{n-1} m_{j\_i} = 1$ ;
28: end for

```

- The substitution of the addition and multiplication: For $a, b, c, d \in \mathbb{F}_2^n$, operations $c = a + b$ and $d = a \times b$ are defined as

ASSERT($c = \text{BVPLUS}(n, a, b)$);
 ASSERT($d = \text{BVMULT}(n, a, b)$);

- The restriction on values: $a \geq b$, $a \leq b$ and $a \neq b$ are defined as

ASSERT($\text{BVGE}(a, b)$);
 ASSERT($\text{BVLE}(a, b)$);
 ASSERT($a \neq b$);

Models for Other Criteria To construct models for a specific criterion, the calculation of which should be explained first. In Algorithm 1, parameter K is taken as the input to restrict the number of allocated gates, and thus the minimal K such that the SAT solver returns a solution successfully is the fewest gates required for implementing the S-box. The criteria shown in Section 2.4 can be calculated as follows:

- Two-qubit Cost: In an NCT-based circuit, only the CNOT gate consumption and the Toffoli gate consumption influence the value of its two-qubit cost. From Table 3 and Eq. (10), among b_0^i, b_1^i and b_2^i , if the current gate is a CNOT gate, only the value of b_1^i equals 1. For the case that the current gate is a Toffoli gate, only the value of b_2^i is equal to 1. Therefore, for an NCT-based circuit with K gates, its two-qubit cost, denoted by T , can be calculated as:

$$T = \sum_{i=1}^K (b_1^i + 5 \times b_2^i). \quad (19)$$

- Quantum Cost: Similar to the calculation of the two-qubit cost of an NCT-based circuit with K gates, denote by Q the quantum cost of the circuit. Then, Q can be calculated as:

$$Q = \sum_{i=1}^K (b_0^i + b_1^i + 5 \times b_2^i). \quad (20)$$

- Toffoli Gate Consumption and Toffoli-Depth: Applying 2 Toffoli gates in parallel requires an NCT-based circuit to have at least 6 qubits, as the qubit cannot be loaded into multiple gates simultaneously. Therefore, for an n -bit circuit with $n \in [3, 5]$, the Toffoli-depth and the Toffoli gate consumption of the circuit are equal. From Eq. (9), the Toffoli gate is marked by b_2^i . Denote by D the Toffoli gate consumption of an NCT-based circuit with K gates. Then, D can be calculated as:

$$D = \sum_{i=1}^K b_2^i. \quad (21)$$

Based on Algorithm 1, the models for designing circuits with criteria two-qubit cost, quantum cost, and the Toffoli gate consumption (as well as the Toffoli-Depth of a small-scale function) can be constructed by adding Eq. (19), Eq. (20), and Eq. (21) in the end of Algorithm 1, respectively, where T , Q , and D are new parameters taken as inputs of the algorithm.

4 Further Insights on Constructing Implementations

In this section, constructing optimized implementations for S-boxes based on the SAT solver is further investigated by taking their cryptographic properties into consideration.

4.1 Optimization based on Affine Equivalence

Applying an invertible affine transformation before and after an S-box does not change the resistance of the S-box against most attacks [34]. In classical application scenarios, if an optimized implementation can be designed for any of the S-boxes affine equivalent to a specific one adopted in a cryptographic cipher, one can replace the specific S-box with the affine equivalent one to improve the implementation performance of the cipher without changing its security. This inspires us to further explore the construction of S-boxes with better implementations for quantum application scenarios based on affine equivalence.

Model of Affine Equivalence To find all the S-boxes affine equivalent to the bijective S-box $S_1 : \mathbb{F}_2^n \rightarrow \mathbb{F}_2^n$, the following issues should be considered:

- The definition of affine equivalence should be satisfied. If the given S-box and the candidate one (denoted by S_2) are affine equivalent, Eq. (5) holds.
- When constructing affine equivalent S-boxes, the transformations A and B searched for Eq. (5) should be linear and be encoded. Specifically, take A as an example, if A is linear, it follows that

$$A(x_1) \oplus A(x_2) = A(x_1 \oplus x_2), \quad (22)$$

where $x_1, x_2 \in \mathbb{F}_2^n$ are inputs of A .

- Both the affine equivalent S-boxes S_1 and S_2 should be bijective, which means that, for any $i, j \in \mathbb{F}_2^n$ and $i \neq j$, we have

$$S_2(i) \neq S_2(j). \quad (23)$$

Model for Implementation With the encoding methods of affine equivalence and the NCT-based circuit, by applying the SAT solver, it is possible to search for an optimal implementation of the S-boxes in the same affine equivalence class with the criterion of gate count. Assuming that the solver outputs a solution when the gate count is set to K , it means that there is an S-box in the affine equivalence class can be implemented with K gates. However, if the solver outputs unsatisfiable, it indicates that none of the S-boxes in the affine equivalence class can be implemented with K gates. At this point, the number of gates should be increased to $K + 1$. The details are illustrated in Algorithm 2.

It should be noted that, in Algorithm 2, S_1 is known, and S_2 is undetermined. After solving the STP model, a corresponding set of solutions will be

Algorithm 2 Implement the S-boxes in the Same Affine Equivalence Class**Require:** Look-up table of the S-box S_1 , the number of gates K **Ensure:** STPModel(S_1, K)

```

1: for  $i = 0$  to  $2^n - 2$  do
2:   for  $j = i$  to  $2^n - 1$  do
3:      $A[i \oplus j] = A[i] \oplus A[j];$  ▷  $A$  and  $B$  are linear
4:      $B[i \oplus j] = B[i] \oplus B[j];$ 
5:   end for
6: end for
7: for  $i = 0$  to  $2^n - 2$  do
8:   for  $j = i$  to  $2^n - 1$  do
9:      $A[i] \neq A[j];$ 
10:     $B[i] \neq B[j];$ 
11:     $S_2[i] \neq S_2[j];$  ▷ All mappings are bijective
12:   end for
13: end for
14: for  $i = 0$  to  $2^n - 1$  do
15:    $S_2[i] = B[S_1[A[i] \oplus a]] \oplus c;$  ▷  $S_2$  is affine equivalent to  $S_1$ 
16: end for
17: STPModel( $S_2, K$ ); ▷ Apply Algorithm 1

```

returned. For the case that both S_1 and S_2 are given, with minor modifications, the algorithm can also be used to determine whether the two S-boxes are in the same equivalence class.

Moreover, in order to identify the optimal implementation scheme of the S-boxes in the same affine equivalence class, the traditional methods require that one should list all the S-boxes in the same affine equivalence class and then construct an STP model for each S-box. By solving all the models, the optimal implementation can be obtained. Due to the numerous affine equivalence of an S-box, the process can be time-consuming. However, by applying Algorithm 2, the affine equivalent S-boxes that satisfy the given conditions can be output directly, as well as their optimized implementations.

4.2 Opimization based on Good Cryptographic Properties

To resist differential and linear attacks, the fundamental cryptographic properties of differential uniformity and linearity are investigated. Additionally, the ability of a cipher to resist differential and linear attacks is influenced by the frequency of occurrence of differential uniformity and linearity, as well as the impact of the BIBO. Therefore, we consider these properties comprehensively. Moreover, previous researches show that the S-box implementation of PRØST [35] outperforms that of other ciphers significantly. The notable difference between the PRØST S-box and other ones lies in the fixed point. Therefore, the fixed point is also taken into consideration in this work.

The implementation of S-boxes based on the above properties can be broadly divided into two parts: one related to the DDT and the other related to the LAT. The calculation of DDT and that of LAT are different, while the calculation of frequency and the calculation of BIBO are the same. In the CVC language, it is necessary to define the number of bits for variables. Moreover, addition is involved both in calculations of BIBO, the frequency of DDT, and the frequency of LAT. If the maximum value that the addition of variables can achieve is ignored, overflow issues may arise. Similarly, subtraction in LAT may result in borrowing. Below, we provide a specific example to explain the computation results in STP.

Example 3 Assuming that variables a , b , and c are defined as 4-bit variables with values $0x1$, $0xF$, and $0x2$, respectively. If there exist two variables $m = a + b$ and $n = a - c$, then m is equal to $0x0$, and n is equal to $0xF$ in the CVC language.

Model for Properties Related to DDT For an $n \times n$ S-box, Lu *et al.* [24] defined DDT as a set of mappings from $\mathbb{F}_2^{2n}$ to $\mathbb{F}_2^n$. However, in this work, the approach in [24] would lead to overflow issues (if a term in the DDT of a 4-bit S-box is 16, then it would be 0 in actual computation), which would result in the actual differential uniformity exceeding the value we specified. To avoid this problem, we define the DDT as a set of mappings from $\mathbb{F}_2^{2n}$ to $\mathbb{F}_2^{n+1}$ and consider the following issues:

- The Calculation of Difference: According to Definition 1, each entry $\text{DDT}[a, b]$, where $a, b \in \mathbb{F}_2^n$, is determined by the number of $x \in \mathbb{F}_2^n$ that satisfies $S(x) \oplus S(x \oplus a) = b$. To determine whether the current x has a difference, we introduce a flag variable, denoted as $\text{IsTrue}[a, b, x]$. If the current x has a difference, $\text{IsTrue}[a, b, x] = 1$, otherwise, $\text{IsTrue}[a, b, x] = 0$. The sum of these flag variables is equal to $\text{DDT}[a, b]$, i.e., $\text{DDT}[a, b] = \text{IsTrue}[a, b, 0] + \dots + \text{IsTrue}[a, b, 2^n - 1]$.
- The Achieving of Differential Uniformity: Constraints are added to ensure that all $\text{DDT}[a, b]$ must be less than or equal to the given differential uniformity $\mathcal{U}$. Additionally, we introduce the constraint $\text{DDT}[1, 1] = \mathcal{U} \text{ OR } \text{DDT}[1, 2] = \mathcal{U} \text{ OR } \dots \text{ OR } \text{DDT}[2^n - 1, 2^n - 1] = \mathcal{U}$ to ensure that the specified differential uniformity is met². This constraint is not presented in the work of Lu *et al.* [24] as they did not independently consider the differential uniformity, but rather combined it with the frequency.
- The Calculation of Frequency: Similar to the calculation of DDT. For a given differential uniformity $\mathcal{U}$, the calculation of the frequency of $\mathcal{U}$ can be achieved by introducing a flag variable for each element $\text{DDT}[a, b]$, denoted by $\text{IsTrue}_{\text{DDT}}[a, b]$, to mark if $\text{DDT}[a, b] = \mathcal{U}$ holds. If $\text{DDT}[a, b] = \mathcal{U}$, $\text{IsTrue}_{\text{DDT}}[a, b] = 1$, otherwise, $\text{IsTrue}_{\text{DDT}}[a, b] = 0$. The frequency of $\mathcal{U}$,

²Without this constraint, when specifying a differential uniformity of 6, the output S-box's differential uniformity might be 4.

denoted by FreqDDT , can be calculated as $\text{FreqDDT} = \text{IsTrue}_{\text{DDT}}[1, 0] + \dots + \text{IsTrue}_{\text{DDT}}[2^n - 1, 2^n - 1]$.

- The Calculation of BIBO: The calculation of BIBO is similar to that of the frequency. However, not all entries in the DDT are required; it only needs to consider the entries with Hamming weight of 1, i.e., $\text{DDT}[a, b]$ with $a, b \in \{1, 2, 4, \dots, 2^{n-1}\}$.

Details are presented in Algorithm 3, where the input parameters are $\mathcal{U}$, $\mathcal{U}_{\text{Freq}}$, and BIBO_{DDT} , representing differential uniformity, the frequency of differential uniformity, and the number of BIBOs in DDT, respectively. We set $\mathcal{U}$ to be equal to a specific value, while $\mathcal{U}_{\text{Freq}}$ and BIBO_{DDT} are set to be less than or equal to the specific values. It is also worth noting that by setting different restrictions on $\mathcal{U}_{\text{Freq}}$ and BIBO_{DDT} , S-boxes with various frequencies and BIBO counts for different levels of differential uniformity can be found, which also means that the parameters $\mathcal{U}_{\text{Freq}}$ and BIBO_{DDT} are optional for Algorithm 3.

Model for Properties Related to LAT The procedure of constructing a model for properties related to LAT is similar to Algorithm 3. Therefore, the details are presented in Appendix A with Algorithm 5.

Model for Implementation With the encoding methods of the S-box property (Algorithms 3 and 5) and the NCT-based circuit (Algorithm 1), combination models can be constructed directly for designing S-boxes and their implementations under specific conditions. To this end, additional constraints need to be added. Firstly, it is essential to ensure that the S-boxes are bijective. Secondly, there is an optional constraint for fixed points³. Since the properties are directly calculated from the S-box, and the NCT-based circuit implementation scheme is represented based on the input and output bits of the S-box, it is necessary to combine them together. For all $i \in \mathbb{F}_2^n$, the input i of the S-box corresponds to $i = x_{i_{n-1}}^0 @ \dots @ x_{i_1}^0 @ x_{i_0}^0$, and the output $S[i]$ corresponds to $S[i] = y_{i_{n-1}} @ \dots @ y_{i_1} @ y_{i_0}$. It implies that the S-box can be implemented by the current NCT-based circuit. The details are presented in Algorithm 4, where constraints on frequencies and BIBO patterns are optional.

Using the SAT solver, solutions that satisfy all the constraint conditions can be output. To obtain all S-boxes that meet the conditions, when STP outputs an S-box, it can be removed from the solution space, and this process can be repeated until the solver outputs no solution.

5 Applications

This section presents the three applications of the proposed models: the first focuses on constructing optimal implementation schemes for S-boxes based on various criteria; the second is to investigate further optimization based on affine equivalence; the last combines the implementation and some typical cryptographic properties of S-boxes.

³It only indicates the absence of fixed point restrictions in the program, implying that the output S-box may or may not have fixed points

Algorithm 3 Properties Related to DDT

Require: Differential uniformity $\mathcal{U}$, frequency of the differential uniformity $\mathcal{U}_{Freq}$, the number of BIBO in DDT BIBO_{DDT} **Ensure:** $\text{STPModel}(\mathcal{U}, \mathcal{U}_{Freq}, \text{BIBO}_{\text{DDT}})$

```

1: for  $a = 0$  to  $2^n - 1$  do
2:   for  $b = 0$  to  $2^n - 1$  do
3:     for  $x = 0$  to  $2^n - 1$  do
4:       if  $S[x \oplus a] = S[x] \oplus b$  then
5:          $\text{IsTrue}[a, b, x] = 1$ ;
6:       else
7:          $\text{IsTrue}[a, b, x] = 0$ ;
8:       end if
9:     end for
10:     $\text{DDT}[a, b] = \text{IsTrue}[a, b, 0] + \dots + \text{IsTrue}[a, b, 2^n - 1]$ ;  $\triangleright$  Definition
    of DDT
11:     $\text{DDT}[a, b] \leq \mathcal{U}$ ;  $\triangleright$  Definition of differential uniformity
12:    if  $\text{DDT}[a, b] = \mathcal{U}$  then
13:       $\text{IsTrue}_{\text{DDT}}[a, b] = 1$ ;
14:    else
15:       $\text{IsTrue}_{\text{DDT}}[a, b] = 0$ ;
16:    end if
17:  end for
18: end for
19:  $\text{DDT}[1, 1] = \mathcal{U} \text{ OR } \dots \text{OR } \text{DDT}[2^n - 1, 2^n - 1] = \mathcal{U}$ ;  $\triangleright$  Ensure  $\mathcal{U}$  is satisfied
20:  $\text{FreqDDT} = \text{IsTrue}_{\text{DDT}}[1, 0] + \text{IsTrue}_{\text{DDT}}[1, 1] + \dots +$ 
    $\text{IsTrue}_{\text{DDT}}[2^{n-1}, 2^{n-1}]$ ;
21:  $\text{FreqDDT} \leq \mathcal{U}_{Freq}$ ;
22:  $Z = [1, 2, 4, \dots, 2^{n-1}]$ ;  $\triangleright$  The set of elements in  $[0, 2^{n-1}]$  with Hamming
   weight 1
23: for  $a \in Z$  do
24:   for  $b \in Z$  do
25:     if  $\text{DDT}[a, b] \neq 0$  then
26:        $\text{BIBO}[a, b] = 1$ ;
27:     else
28:        $\text{BIBO}[a, b] = 0$ ;
29:     end if
30:   end for
31: end for
32:  $\text{BIBODDT} = \text{BIBO}[1, 1] + \text{BIBO}[1, 2] + \dots + \text{BIBO}[2^{n-1}, 2^{n-1}]$ ;  $\triangleright$ 
   Definition of BIBO
33:  $\text{BIBODDT} \leq \text{BIBO}_{\text{DDT}}$ ;

```

Algorithm 4 Combine the S-box Circuit and Cryptographic Properties

Require: The number of gates K , differential uniformity $\mathcal{U}$, linearity $\mathcal{L}$, frequency of $\mathcal{U}/\mathcal{L}$ $\mathcal{U}_{Freq}/\mathcal{L}_{Freq}$, the number of BIBO in DDT/LAT $\text{BIBO}_{\text{DDT}}/\text{BIBO}_{\text{LAT}}$

Ensure: $\text{STPModel}(K, \mathcal{U}, \mathcal{L}, \mathcal{U}_{Freq}, \mathcal{L}_{Freq}, \text{BIBO}_{\text{DDT}}, \text{BIBO}_{\text{LAT}})$

```

1: for  $i = 0$  to  $2^n - 1$  do
2:    $S[i] \neq i$ ; ▷ The S-box  $S$  has no fixed points (optional)
3: end for
4: for  $i = 0$  to  $2^n - 2$  do
5:   for  $j = i$  to  $2^n - 1$  do
6:      $S[j] \neq S[i]$ ; ▷ The S-box  $S$  is a bijection
7:   end for
8: end for
9:  $\text{STPModel}(\mathcal{U}, \mathcal{U}_{Freq}, \text{BIBO}_{\text{DDT}})$ ; ▷ Apply Algorithm 3
10:  $\text{STPModel}(\mathcal{L}, \mathcal{L}_{Freq}, \text{BIBO}_{\text{LAT}})$ ; ▷ Apply Algorithm 5
11: for  $i = 0$  to  $2^n - 1$  do
12:    $i = x_{i_{n-1}}^0 @ \dots @ x_{i_1}^0 @ x_{i_0}^0$ ;
13:    $S[i] = y_{i_{n-1}} @ \dots @ y_{i_1} @ y_{i_0}$ ;
14: end for
15:  $\text{STPModel}(S, K)$ ; ▷ Apply Algorithm 1

```

5.1 Optimizing the S-box Implementation with Various Criteria

To construct optimized NCT-based circuits for small S-boxes, Dasu *et al.* [18] assigned the NCT gates with different cost metrics of gate count, two-qubit cost and quantum cost, which also be adopted as criteria in our models in this section.

Application to 4-bit S-boxes Based on Algorithm 1, Eqs. (19) and (20), we construct models to search for optimal implementation schemes for the 4-bit S-boxes adopted in some well-known cryptographic ciphers [35–41], the look-up tables of which are listed in Appendix B. To explain the transformation from the output of the SAT solver to an NCT-based circuit, the S-box of GIFT [41] is taken as the input of Algorithm 1 and the following example is given.

Example 4 Denote by (x_0, x_1, x_2, x_3) and (y_0, y_1, y_2, y_3) the input and the output of the GIFT S-box, when the number of logic gates is set to 8, the solver outputs the following solution:

$$\begin{array}{llll}
0 \ x_0^0 = x_0, & 4 \ x_1^1 = x_1^0 + x_0^0 \cdot x_2^0, & 8 \ x_1^5 = x_1^4 + 1, & 12 \ y_0 = x_1^8, \\
1 \ x_1^0 = x_1, & 5 \ x_0^2 = x_0^1 + x_1^1 \cdot x_3^1, & 9 \ x_3^6 = x_3^5 + x_0^5 \cdot x_1^5, & 13 \ y_1 = x_3^8, \\
2 \ x_2^0 = x_2, & 6 \ x_3^3 = x_3^2 + x_2^2, & 10 \ x_1^7 = x_1^6 + x_3^6, & 14 \ y_2 = x_2^8, \\
3 \ x_3^0 = x_3, & 7 \ x_2^4 = x_2^3 + x_1^3, & 11 \ x_2^8 = x_2^7 + x_0^7 \cdot x_1^7, & 15 \ y_3 = x_0^8.
\end{array}$$

The quantum implementation corresponds to the above operations is shown in Figure 3.

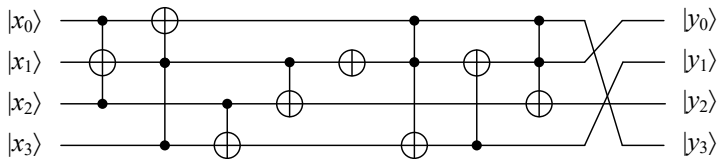


Fig. 3 NCT-based circuit of the GIFT S-box.

The experimental results reveal that optimal solutions can be obtained for all tested 4-bit S-boxes. The details are shown in Table 4, where GC, 2qC and QC represent the criterion gate count, two-qubit cost, and quantum cost, respectively. Additionally, Table 4 also provides cryptographic properties of each S-box. Notably, apart from the GIFT S-box, all S-boxes share the same properties listed in the table except BIBO.

Table 4 Implementations of the 4-bit S-boxes

S-box	$\mathcal{U}$	$\mathcal{L}$	$\mathcal{U}_{Freq}$	$\mathcal{L}_{Freq}$	BIBO _{DDT}	BIBO _{LAT}	GC	2qC	QC
PRØST	4	8	24	36	5	8	4	20	20
PICCOLO	4	8	24	36	4	7	9	20	25
SKINNY	4	8	24	36	4	7	10	20	26
PRESENT	4	8	24	36	0	8	11	25	27
RECTANGLE	4	8	24	36	2	2	10	23	26
LAC	4	8	24	36	4	6	8	22	24
GIFT	6	8	2	36	1	3	8	22	24

Moreover, we apply the tool LIGHTER-R to the S-boxes listed in Table 4, and the results reveal that, for each criterion, implementations of those S-boxes returned by LIGHTER-R are also optimal.

In general, the search for the optimal solution based on the SAT solver is considered to be time-consuming. However, in our experiments, the search procedures of all the models constructed based on various criteria consume only a few seconds for most S-boxes (less than 0.1s for the PRØST S-box).

Application to 5-bit S-boxes Constructing an optimal implementation of a 5-bit S-box is often considered tricky due to the high complexity. For this reason, the tool LIGHTER-R can only be used to construct NCT-based circuits for small-scale S-boxes, such as the 3-/4-bit ones. Although the solver may fail to output any valid solutions within a reasonable time for a 5-bit S-box, with an appropriate value of the criterion assigned to the model, the solver responds quickly. As a consequence, Algorithm 1, as well as that combined with Eqs. (19) - (21), can be used to provide a lower bound of a certain criterion for implementing a 5-bit S-box. In this case, we take the criterion gate count as an example and take the 5-bit S-boxes of ASCON [42] and KECCAK [43] as the inputs of the models, the results are shown in Table 5, where GC represents the gate count.

Table 5 Implementations of the 5-bit S-boxes

S-box	$\mathcal{U}$	$\mathcal{L}$	$\mathcal{U}_{Freq}$	$\mathcal{L}_{Freq}$	BIBO _{DDT}	BIBO _{LAT}	GC
ASCON	8	16	20	40	0	0	>12
KECCAK	8	16	20	40	5	5	>9

5.2 Constructing S-box Implementations based on Affine Equivalence

Both the look-up table of an S-box and a representation of an affine equivalence class can be taken as the input of Algorithm 2, in which Algorithm 1 is embedded. Recall that the models for criteria gate count, two-qubit cost, and quantum cost can all be constructed based on Algorithm 1. In this section, the criterion gate count will be taken as an example to exhibit the performance of the proposed models. We initialize the number of allocated gates K to 1 and construct models. If the SAT solver does not output a solution for a long time, we increase the value of K by 1 and repeat the procedure until the solver outputs a solution.

Taking S-boxes as Inputs As did in Section 5.1, we take the S-boxes adopted in [35–41] as the inputs of the models. The results are listed in Table 6, where GC represents the criterion gate count. Notably, no restrictions of the fixed point are imposed on the models for Table 6. For the case that the fixed point is not allowed, the results are listed in Table 7.

Table 6 Results based on affine equivalence

S-box	GC	S-box in the affine equivalence class	GC
PRØST	4	the PRØST S-box	4*
PICCOLO	9	0, 8, 4, 14, 1, 15, 5, 9, 2, 10, 7, 11, 13, 3, 12, 6	4*
SKINNY	10	0, 8, 1, 7, 2, 13, 3, 10, 4, 12, 15, 9, 14, 11, 5, 6	4*
PRESENT	11	0, 1, 8, 7, 4, 13, 12, 11, 2, 3, 9, 14, 10, 5, 15, 6	5†
RECTANGLE	10	0, 8, 2, 14, 1, 15, 9, 3, 12, 7, 10, 5, 13, 4, 6, 11	5†
LAC	8	0, 1, 8, 9, 4, 13, 14, 11, 2, 15, 7, 10, 6, 3, 5, 12	4*
GIFT	8	0, 8, 1, 15, 2, 14, 3, 9, 4, 13, 10, 5, 7, 12, 11, 6	5†

★ Optimal.

† No solution is returned in 1 day if the gate count is reduced by 1.

From Table 6, without the restrictions on the fixed point, there is always an S-box in the class that can be implemented with only 4 or 5 gates. Compared to the optimal circuits designed for the given S-boxes (listed in the second column of Table 6), except that for the PRØST S-box, the gate consumption of each new one is reduced by half or more. For the case that no fixed point is allowed, it can be observed from Table 7 that one more gate is required by each S-box.

Taking Affine Equivalence Classes as Inputs For 4-bit S-boxes, there are 302 affine equivalence classes. As pointed out in [34], 16 classes are optimal (listed in Table C1 in Appendix C). Among those classes,

Table 7 Results based on affine equivalence with no fixed point is allowed

S-box	GC	S-box in the affine equivalence class	GC
PRØST	4	2, 0, 15, 8, 3, 1, 10, 13, 14, 4, 5, 12, 9, 7, 6, 11	5 [†]
PICCOLO	9	8, 12, 7, 2, 0, 4, 10, 13, 11, 3, 6, 15, 1, 14, 9, 5	5 [*]
SKINNY	10	2, 7, 6, 13, 15, 10, 12, 3, 0, 1, 4, 14, 8, 9, 11, 5	5 [†]
PRESENT	11	8, 12, 7, 2, 0, 4, 10, 13, 11, 3, 6, 15, 1, 14, 9, 5	6 [†]
RECTANGLE	10	4, 0, 5, 8, 6, 3, 10, 14, 13, 1, 11, 7, 15, 2, 12, 9	6 [†]
LAC	8	8, 10, 0, 13, 9, 11, 7, 2, 12, 6, 4, 3, 15, 5, 1, 14	5 [†]
GIFT	8	2, 3, 4, 9, 0, 1, 7, 14, 10, 15, 12, 5, 11, 6, 8, 13	6 [†]

★ Optimal.

† No solution is returned in 1 day if the gate count is reduced by 1.

$G_0, G_1, G_2, G_4, G_5, G_7, G_8$ and G_{13} are even, that is, they can be implemented without introducing additional ancilla qubits. The S-boxes in PRØST, PICCOLO, SKINNY, and LAC are in G_8 . From Table 6, the implementation of the PRØST S-box is optimal in the class. As a result, there is no need to deal with the class G_8 . Similarly, both the S-boxes of PRESENT and RECTANGLE are in G_1 , which can also be ignored, as one can deduce from Table 6 that the implementation returned by our model consumes at least 5 gates for the S-boxes in G_1 . The optimal implementations with criterion gate count of the remaining classes returned by our models are shown in Table 8. Besides, we also construct models for the case that the fixed point is not allowed, the results are listed in Table 9.

Table 8 Results based on affine equivalence classes

S-box	representation	GC
G_0	0, 4, 2, 6, 1, 15, 7, 12, 8, 14, 13, 11, 9, 5, 10, 3	4 [*]
G_1	0, 8, 2, 14, 1, 15, 9, 3, 12, 7, 10, 5, 13, 4, 6, 11	5 [†]
G_2	0, 1, 8, 14, 4, 7, 12, 9, 2, 3, 11, 5, 15, 13, 6, 10	4 [*]
G_4	0, 9, 2, 10, 1, 8, 7, 6, 4, 15, 14, 5, 13, 11, 12, 3	7 [†]
G_5	0, 4, 2, 15, 1, 11, 9, 14, 8, 6, 10, 5, 13, 12, 7, 3	7 [†]
G_7	0, 13, 1, 9, 2, 10, 7, 12, 4, 8, 15, 14, 11, 6, 5, 3	7 [†]
G_8	the PRØST S-box	4 [*]
G_{13}	0, 4, 2, 10, 8, 12, 13, 7, 1, 15, 3, 5, 11, 14, 6, 9	7 [†]

★ Optimal.

† No solution is returned in 1 day if the gate count is reduced by 1.

The results listed in Tables 8 - 9 show that NCT-based circuits of the S-boxes in $G_0, G_1, G_2, G_4, G_5, G_7, G_8, G_{13}$ consume at least 4 gates if no restrictions are imposed on the fixed point. However, with the restriction that the fixed point is not allowed, one more gate is required by each S-box.

Table 9 Results based on affine equivalence classes with no fixed point is allowed

S-box	representation	GC
G_0	8, 11, 12, 5, 15, 6, 10, 9, 0, 1, 4, 14, 2, 3, 7, 13	5*
G_1	8, 12, 7, 2, 0, 4, 10, 13, 11, 3, 6, 15, 1, 14, 9, 5	6†
G_2	4, 0, 5, 11, 6, 3, 7, 10, 12, 8, 15, 1, 9, 14, 2, 13	5*
G_4	1, 9, 15, 10, 5, 12, 11, 13, 0, 8, 2, 14, 7, 6, 3, 4	7†
G_5	2, 10, 0, 5, 3, 13, 1, 14, 6, 12, 11, 8, 4, 9, 15, 7	8†
G_7	4, 6, 15, 14, 5, 9, 10, 13, 0, 3, 8, 12, 11, 1, 2, 7	8†
G_8	2, 0, 15, 8, 3, 1, 10, 13, 14, 4, 5, 12, 9, 7, 6, 11	5†
G_{13}	1, 0, 9, 11, 7, 8, 3, 12, 13, 4, 5, 14, 6, 15, 10, 2	8†

* Optimal.

† No solution is returned in 1 day if the gate count is reduced by 1.

5.3 Combining Implementations and Cryptographic Properties of S-boxes

Similar to Section 5.2, the criterion gate count will be taken as an example in this section to exhibit the performance of the proposed models.

Application to 4-bit S-boxes The optimal differential uniformity and linearity of 4-bit S-boxes is 4 and 8 respectively [34]. Therefore, we set $\mathcal{U} = 4$ and $\mathcal{L} = 8$ as the inputs of Algorithm 4 and apply the model to search for S-boxes with fixed points. In this case, the solver returns 3456 solutions until $K = 4$. To construct more S-boxes with various cryptographic properties, we then construct three kinds of models: the first is constructed by fixing the differential uniformity, linearity and frequency as that of the GIFT S-box; the second is constructed as did in [14] by restricting that $\mathcal{U} = 4$, $\mathcal{L} = 8$, $\mathcal{U}_{Freq} \leq 15$ and $\mathcal{L}_{Freq} \leq 30$; the last takes all the properties into consideration. Models for the case that the fixed point is not allowed can be constructed similarly. The application results of the models are listed in Table 10⁴, where ‘-’ indicates no constraints on the current properties.

Table 10 Application to 4-bit S-boxes

$\mathcal{U}$	$\mathcal{L}$	$\mathcal{U}_{Freq}$	$\mathcal{L}_{Freq}$	BIBO _{DDT}	BIBO _{LAT}	GC	S-box
With Fixed Points							
4	8	-	-	-	-	4*	{3456}
6	8	2	36	-	-	5*	✓
4	8	15	30	-	-	6†	✓
4	8	24	36	4	6	4*	✓
Without Fixed Points							
4	8	-	-	-	-	5*	✓
4	8	15	30	-	-	7*	✓
4	8	24	36	4	4	7†	✓

* Optimal.

† No solution is returned in 1 day if the gate count is reduced by 1.

⁴Appendix D also presents some S-boxes obtained by assigning various values to different properties.

With the restriction that the fixed point is allowed, all the solutions can be returned in a reasonable time, which gives rise to the following theorem.

Theorem 1 *The minimal number of NCT gates for implementing 4-bit optimal S-boxes corresponding to even permutations is 4. There are 3456 such optimal S-boxes, and each of them has at least one fixed point ($S[0] = 0$).*

Proof To proof the minimal number of NCT gates for implementing 4-bit optimal S-boxes corresponding to even permutations is 4, the first step is to verify that no optimal S-boxes can be built if no more than 3 gates are allowed, i.e., $K \leq 3$, whileas the second step is to verify that an optimal S-box can be built with 4 gates.

Denote by $\text{Cir}_{n_t, n_c, n_x}$ the circuit build with n_t Toffoli gates, n_c CNOT gates and n_x Pauli-X gates. Note that $\text{Cir}_{n_t, n_c, n_x}$ is a set of circuits, since different gate application orders mean different circuits.

Algebraic degree is invariant under affine equivalence [34]. The algebraic degree of each representation in 16 affine equivalence classes is 3, it implies that at least 2 Toffoli gates are required for implementing such S-boxes, i.e., $n_t \geq 2$.

For $2 \leq K \leq 3$, candidate S-box circuits are $\text{Cir}_{2,0,0}$, $\text{Cir}_{2,0,1}$, $\text{Cir}_{2,1,0}$ and $\text{Cir}_{3,0,0}$. The CNOT gate in $\text{Cir}_{2,1,0}$, and the Pauli-X gate in $\text{Cir}_{2,0,1}$ can be applied before, between or after two Toffoli gates. Theoretically, there are $3 \cdot C_4^2 \cdot A_2^2 \cdot C_4^3 \cdot A_3^3 \cdot C_4^3 \cdot A_3^3 = 20736$ implementation schemes to build S-boxes with $\text{Cir}_{2,1,0}$ and $3 \cdot C_4^1 \cdot C_4^3 \cdot A_3^3 \cdot C_4^3 \cdot A_3^3 = 6912$ implementation schemes to build S-boxes with $\text{Cir}_{2,0,1}$.

However, far few possible circuits need to be considered. For instance, affine equivalent S-boxes share differential uniformity and linearity, this helps to reduce the number of candidate circuits. The analysis details are presented in Appendix E.

By evaluating the differential uniformity and linearity of the permutations correspond to instances in all the remaining candidate circuits, the number of optimal 4-bit S-boxes can be obtained. The results reveal that no optimal S-boxes are returned if $2 \leq K \leq 3$.

For $K = 4$, candidate circuits are $\text{Cir}_{2,0,2}$, $\text{Cir}_{2,2,0}$, $\text{Cir}_{2,1,1}$, $\text{Cir}_{3,0,1}$, $\text{Cir}_{3,1,0}$ and $\text{Cir}_{4,0,0}$. The reduction of candidate circuits is presented in Appendix E. Ultimately, for $K = 4$, only $\text{Cir}_{4,0,0}$ provides 3456 optimal S-boxes. At this point, the mapping from 0 to 0 always holds. $\square$

Note that when constructing the above models, we only take the total number of allocated gates (i.e., K) as the input of the algorithms, no additional constraints are placed on gates. In fact, we can also limit the Toffoli gate count allocated for the models (by Eq. (21)), from which the Toffoli-Depth of the circuit can also be derived. Combined with our experiments with restriction on the Toffoli gate consumption and the results listed in Table 10, the following theorem is presented.

Theorem 2 *For all optimal 4-bit S-boxes corresponding to even permutations, with or without fixed points, the NCT-based implementation requires at least 4 Toffoli gates. This also implies that, unless additional ancilla qubits are introduced, the lowest Toffoli-depth of the NCT-based circuit of any optimal 4-bit S-box is 4.*

Proof The case that fixed points are allowed can be directly derived from the proof of Theorem 1, which also demonstrates that the implementation of optimal S-boxes without fixed points corresponding to even permutations requires that $K \geq 5$. Thus, for the case that optimal S-boxes without fixed points, the first step is to verify that it is impossible to construct optimal S-boxes without fixed points when $n_t \leq 3$; the second step is to verify that there exist optimal S-boxes without fixed points when $n_t = 4$.

For $n_t \leq 3$, from the proof of Theorem 1, the candidate circuits are $\text{Cir}_{2,i_0,0} = \{\mathbf{T} \circ \mathbf{C}^{i_0} \circ \mathbf{T}\}$ and $\text{Cir}_{3,i_1+i_2,0} = \{\mathbf{T} \circ \mathbf{C}^{i_1} \circ \mathbf{T} \circ \mathbf{C}^{i_2} \circ \mathbf{T}\}$, where $i_0, i_1, i_2 \in [0, +\infty)$. Note that the application of CNOT gates fundamentally constitutes matrix multiplication, and the number of 4×4 invertible matrices over $\mathbb{F}_2$ is finite, limiting the number of S-boxes derived from the circuits in $\{\mathbf{T} \circ \mathbf{C}^{i_0} \circ \mathbf{T}, \mathbf{T} \circ \mathbf{C}^{i_1} \circ \mathbf{T} \circ \mathbf{C}^{i_2} \circ \mathbf{T}\}$. However, no optimal S-boxes can be built in this case.

For $n_t = 4$, from the view of affine equivalence, the candidate circuit $\text{Cir}_{4,i_0+i_1+i_2,0} = \{\mathbf{T} \circ \mathbf{C}^{i_0} \circ \mathbf{T} \circ \mathbf{C}^{i_1} \circ \mathbf{T} \circ \mathbf{C}^{i_2} \circ \mathbf{T}\}$. However, circuits built only with Toffoli gates and CNOT gates must have a fixed point 0, the candidate circuits we should verify are in $\{\mathbf{X}^{j_0} \circ \mathbf{T} \circ \mathbf{X}^{j_1} \circ \mathbf{C}^{i_0} \circ \mathbf{X}^{j_2} \circ \mathbf{T} \circ \mathbf{X}^{j_3} \circ \mathbf{C}^{i_1} \circ \mathbf{X}^{j_4} \circ \mathbf{T} \circ \mathbf{X}^{j_5} \circ \mathbf{C}^{i_2} \circ \mathbf{X}^{j_6} \circ \mathbf{T} \circ \mathbf{X}^{j_7}\}$. To proof the existence, we can start with one Pauli-X gate is allowed (i.e., $n_x = j_0 + j_1 + \dots + j_7 = 1$) and increase n_x until an optimal S-box without fixed points can be obtained by tested circuits. Fortunately, it holds when $\text{Cir}_{4,0,1} = \{\mathbf{X} \circ \mathbf{T}^4\}$ is verified at the very beginning.

Based on the above analysis, the fewest Toffoli-depth of the optimal 4-bit S-box circuits is 4 can be deduced⁵.

□

Application to 5-bit S-boxes Although the solver cannot find optimal solutions for existing 5-bit S-boxes in a reasonable time. However, using the proposed models, it is possible to construct optimized implementations of S-boxes that are equivalent or superior to existing ones in cryptographic properties. As a consequence, similar to the construction of the models for 4-bit S-boxes, we construct different models for 5-bit S-boxes by fixing various properties. Note that when considering all the properties, we set the cryptographic properties input to the models the same as that of the KECCAK S-box. The application results of those models are listed in Table 11⁶.

Compared with the result listed in Table 5 that the gate consumption of the KECCAK S-box implementation is more than 9, the application of our proposed models reveals that we can construct S-boxes with the same properties as the KECCAK S-box but consume only 5 gates.

6 Conclusion

In this work, the optimization of the NCT-based circuits for S-boxes based on the SAT solver is investigated. To this end, we proposed encoding methods

⁵The verification codes of Theorem 2 are available at <https://github.com/damash/SAT-based-methods-of-Sbox-qcircuits>.

⁶Appendix D also presents some S-boxes obtained by assigning various values to different properties.

Table 11 Application to 5-bit S-boxes

$\mathcal{U}$	$\mathcal{L}$	$\mathcal{U}_{Freq}$	$\mathcal{L}_{Freq}$	BIBO _{DDT}	BIBO _{LAT}	GC	S-box
With Fixed Points							
8	16	-	-	-	-	5	✓
2	8	-	-	-	-	[5,9]	×
8	16	10	30	-	-	6	✓
8	16	20	40	5	5	5 [†]	✓
Without Fixed Points							
8	16	-	-	-	-	6	✓
8	16	10	30	-	-	7	✓
8	16	20	40	5	5	6	✓

† The cryptographic properties are similar to that of the KECCAK S-box.

for the NCT-based circuits and some important cryptographic properties of S-boxes. Based on the proposed methods, we constructed various STP models to identify the optimal implementation schemes of S-boxes with different criteria, to further investigate optimizing the S-box implementations based on affine equivalence, and to search for S-boxes that match the given restrictions on the implementation and properties. Since the inputs of the models proposed in this work are tweakable, apart from applications of the models to search for S-boxes that match given criterion configurations, one can also apply the models with restrictions placed on the number of each logic gate and the Toffoli-depth. It should be noted that the above functions are not currently available in the public tools for constructing optimized NCT-based circuits for S-boxes.

For the purpose of saving qubits, only the implementations of even permutations are considered in this work. If more ancilla qubits are allowed, NCT-based circuits for odd permutations can be designed. Besides, it is widely believed that applying ancilla qubits may help to optimize the performance of S-box circuits in criteria such as Depth. We leave the solution of the above problems based on the SAT solver for the case that more ancilla qubits are available as possible direction for future research.

Statements and Declarations

Funding

Da Lin was supported by the National Natural Science Foundation of China (No. 62402511), Chunli Yang and Shizhu Tian were supported by the National Natural Science Foundation of China under Grant 62102135 and Wuhan Science and Technology Bureau (No. 2023010201020420), Shengyuan Xu was supported by Shandong Provincial Natural Science Foundation (No. ZR2024QA205), Bing Sun was supported by the National Natural Science Foundation of China (No. 62272470).

Competing interests

The authors have no competing interests to declare that are relevant to the content of this article.

Author contributions

The authors equally contributed to the paper.

Data availability

The datasets generated during and/or analysed during the current study are available at <https://github.com/damash/SAT-based-methods-of-Sbox-qcircuits>.

Appendix A The Model for LAT-related Properties

The calculation of LAT is similar to that of DDT, with the difference that negative values may appear in LAT. However, the CVC language does not support the representation of negative numbers. Therefore, we manually take the absolute value of each entry in LAT, which is called ABLAT. From Definition 2, $\text{LAT}[a, b] = \text{LATT}[a, b] - 2^{n-1}$, where $a, b \in \mathbb{F}_2^n$ and $\text{LATT}[a, b] = \#\{x \in \mathbb{F}_2^m \mid a \cdot x \oplus b \cdot S(x) = 0\}$. If $\text{LATT}[a, b] \geq 2^{n-1}$, then $0 \leq \text{LAT}[a, b] \leq 2^{n-1}$, and $\text{ABLAT}[a, b] = \text{LAT}[a, b]$. However, if $\text{LATT}[a, b] \leq 2^{n-1}$, carry out borrowing will occur, as illustrated in Example 3. In this case, $\text{LAT} = \text{LATT}[a, b] + 2^n - 2^{n-1} = \text{LATT}[a, b] + 2^{n-1}$, $2^{n-1} \leq \text{LAT}[a, b] \leq 2^n$, the absolute value is given by $\text{ABLAT}[a, b] = 2^n - \text{LAT}[a, b]$. The calculation of frequency and the calculation of BIBO in LAT are similar to those in DDT, and we have omitted this process in Algorithm 5.

Appendix B Look-up Tables of the Related S-boxes

PRESENT: [12, 5, 6, 11, 9, 0, 10, 13, 3, 14, 15, 8, 4, 7, 1, 2]

GIFT: [1, 10, 4, 12, 6, 15, 3, 9, 2, 13, 11, 7, 5, 0, 8, 14]

PICCOLO: [14, 4, 11, 2, 3, 8, 0, 9, 1, 10, 7, 15, 6, 12, 5, 13]

LAC: [14, 9, 15, 0, 13, 4, 10, 11, 1, 2, 8, 3, 7, 6, 12, 5]

PRØST: [0, 4, 8, 15, 1, 5, 14, 9, 2, 7, 10, 12, 11, 13, 6, 3]

RECTANGLE: [6, 5, 12, 10, 1, 14, 7, 9, 11, 0, 3, 13, 8, 15, 4, 2]

SKINNY: [12, 6, 9, 0, 1, 10, 2, 11, 3, 8, 5, 13, 4, 14, 7, 15] 22, 12, 11, 19]

ASCON: [4, 11, 31, 20, 26, 21, 9, 2, 27, 5, 8, 18, 29, 3, 6, 28, 30, 19, 7, 14, 0, 13, 17, 24, 16, 12, 1, 25, 22, 10, 15, 23]

KECCAK: [0, 5, 10, 11, 20, 17, 22, 23, 9, 12, 3, 2, 13, 8, 15, 14, 18, 21, 24, 27, 6, 1, 4, 7, 26, 29, 16, 19, 30, 25, 28, 31]

Algorithm 5 Properties Related to LAT**Require:** Linearity $\mathcal{L}$ **Ensure:** STPModel($\mathcal{L}$)

```

1: for  $a = 0$  to  $2^n - 1$  do
2:   for  $b = 0$  to  $2^n - 1$  do
3:     for  $x = 0$  to  $2^n - 1$  do
4:       if  $a \cdot x = b \cdot S[x]$  then
5:         IsTrueLAT[ $a, b, x$ ] = 1;
6:       else
7:         IsTrueLAT[ $a, b, x$ ] = 0;
8:       end if
9:     end for
10:    LAT[ $a, b$ ] = LATT[ $a, b$ ] -  $2^{n-1}$ ; ▷ Definition of LAT
11:    LATT[ $a, b$ ] = IsTrueLAT[ $a, b, 0$ ] +  $\dots$  + IsTrueLAT[ $a, b, 2^{n-1}$ ];
12:    if LAT[ $a, b$ ]  $\geq 2^{n-1}$  then
13:      ABLAT[ $a, b$ ] =  $2^n - \text{LAT}[a, b]$ ;
14:    else
15:      ABLAT[ $a, b$ ] = LAT[ $a, b$ ]; ▷ Definition of ABLAT
16:    end if
17:    ABLAT[ $a, b$ ]  $\leq \mathcal{L}$ ; ▷ Definition of linearity
18:  end for
19: end for

```

Appendix C Affine Equivalence Classes with $\mathcal{U} = 4$ and $\mathcal{L} = 8$

Table C1 Affine equivalence classes

Equivalence class	representative
G_0	0, 1, 2, 13, 4, 7, 15, 6, 8, 11, 12, 9, 3, 14, 10, 5
G_1	0, 1, 2, 13, 4, 7, 15, 6, 8, 11, 14, 3, 5, 9, 10, 12
G_2	0, 1, 2, 13, 4, 7, 15, 6, 8, 11, 14, 3, 10, 12, 5, 9
G_3	0, 1, 2, 13, 4, 7, 15, 6, 8, 12, 5, 3, 10, 14, 11, 9
G_4	0, 1, 2, 13, 4, 7, 15, 6, 8, 12, 9, 11, 10, 14, 5, 3
G_5	0, 1, 2, 13, 4, 7, 15, 6, 8, 12, 11, 9, 10, 14, 3, 5
G_6	0, 1, 2, 13, 4, 7, 15, 6, 8, 12, 11, 9, 10, 14, 5, 3
G_7	0, 1, 2, 13, 4, 7, 15, 6, 8, 12, 14, 11, 10, 9, 3, 5
G_8	0, 1, 2, 13, 4, 7, 15, 6, 8, 14, 9, 5, 10, 11, 3, 12
G_9	0, 1, 2, 13, 4, 7, 15, 6, 8, 14, 11, 3, 5, 9, 10, 12
G_{10}	0, 1, 2, 13, 4, 7, 15, 6, 8, 14, 11, 5, 10, 9, 3, 12
G_{11}	0, 1, 2, 13, 4, 7, 15, 6, 8, 14, 11, 10, 5, 9, 12, 3
G_{12}	0, 1, 2, 13, 4, 7, 15, 6, 8, 14, 11, 10, 9, 3, 12, 5
G_{13}	0, 1, 2, 13, 4, 7, 15, 6, 8, 14, 12, 9, 5, 11, 10, 3
G_{14}	0, 1, 2, 13, 4, 7, 15, 6, 8, 14, 12, 11, 3, 9, 5, 10
G_{15}	0, 1, 2, 13, 4, 7, 15, 6, 8, 14, 12, 11, 9, 3, 10, 5

Appendix D Results of the S-boxes with Various Restrictions on Properties

Table D2 S-boxes obtained by placing various restraints on properties

$\mathcal{U}$	$\mathcal{L}$	$\mathcal{U}_{Freq}$	$\mathcal{L}_{Freq}$	BIBO _{DDT}	BIBO _{LAT}	GC	S-box
4	8	24	36	4	6	4	S_1
4	8	15	30	9	12	6	S_2
6	8	2	36	8	8	5	S_3
4	8	24	36	4	7	5	S_4
4	8	24	36	4	4	7	S_5
4	8	15	30	9	12	7	S_6
8	16	20	40	5	5	5	S_7
8	16	7	28	8	10	6	S_8
8	16	20	40	5	5	6	S_9
8	16	8	23	8	14	7	S_{10}

S_1 : [0, 8, 4, 14, 2, 10, 7, 13, 1, 15, 5, 9, 12, 3, 11, 6]

S_2 : [0, 4, 1, 5, 8, 11, 3, 9, 2, 14, 15, 6, 10, 7, 12, 13]

S_3 : [0, 8, 4, 7, 1, 11, 13, 12, 2, 6, 5, 10, 3, 15, 14, 9]

S_4 : [1, 0, 9, 8, 3, 10, 12, 15, 5, 14, 6, 13, 7, 4, 11, 2]

S_5 : [4, 7, 0, 9, 6, 12, 8, 10, 13, 2, 11, 14, 15, 1, 3, 5]

S_6 : [4, 5, 0, 1, 12, 3, 10, 8, 6, 15, 7, 13, 14, 11, 9, 2]

S_7 : [0, 8, 16, 24, 4, 12, 21, 29, 2, 31, 23, 10, 6, 26, 18, 14, 1, 22, 17, 11, 13, 27, 28, 7, 3, 9, 30, 25, 15, 5, 19, 20]

S_8 : [0, 1, 16, 31, 4, 13, 20, 22, 8, 9, 10, 21, 12, 5, 14, 28, 2, 3, 18, 29, 15, 6, 19, 17, 26, 23, 24, 11, 27, 30, 25, 7]

S_9 : [1, 5, 0, 6, 11, 15, 2, 4, 9, 13, 8, 26, 23, 19, 30, 12, 17, 22, 16, 21, 27, 20, 18, 31, 29, 14, 28, 25, 3, 24, 10, 7]

S_{10} : [2, 6, 10, 12, 7, 3, 30, 29, 0, 4, 8, 14, 1, 5, 25, 26, 27, 28, 19, 31, 13, 11, 21, 15, 16, 23, 24, 20, 17, 22, 9, 18]

Appendix E Reduction of Candidate Circuits

According to [26] (Theorem 29), the Pauli-X gates in an NCT-based circuit can be moved to the end of the circuit, at the cost of increasing CNOT gate consumption. Denote by **T**, **C** and **X** the applications of Toffoli gate, CNOT gate and Pauli-X gate, respectively. Any NCT-based circuit with Toffoli-depth d can be represented as $\text{Cir}_{n_t, n_c, n_x} = \mathbf{C}^{i_0} \circ \mathbf{T}^{j_0} \circ \mathbf{C}^{i_1} \circ \mathbf{T}^{j_1} \circ \dots \circ \mathbf{C}^{i_{d-1}} \circ \mathbf{T}^{j_{d-1}} \circ \mathbf{C}^{i_d} \circ \mathbf{X}^{i_{d+1}}$, where $\mathbf{C}^i$, $\mathbf{T}^i$, $\mathbf{C}^i$ denote the sub-circuit consisting of i CNOT gates, Toffoli gates and Pauli-X gates, respectively.

Lemma 1 Let S be an S-box from $\mathbb{F}_2^4$ to $\mathbb{F}_2^4$, $\text{Cir}_{n_t, n_c, n_x}$ be an NCT-based circuit with Toffoli-depth d of S . Then, S is affine equivalent to the S-box obtained by $\text{Cir}'_{n_t, n_c, n_x} = T \circ C^{i_1} \circ T \circ \dots \circ C^{i_{d-1}} \circ T$.

Proof The Toffoli gates in the NCT-based circuit of a 4-bit S-box must be applied serially; that is, each Toffoli layer contains only one Toffoli gate. Consequently, $\text{Cir}_{n_t, n_c, n_x}$ can be represented as $\text{Cir}_{n_t, n_c, n_x} = C^{i_0} \circ T \circ C^{i_1} \circ T \circ \dots \circ C^{i_{d-1}} \circ T \circ C^{i_d} \circ X^{i_{d+1}}$. Here, C^{i_0} forms a linear transformation acting on S-box inputs, while $C^{i_d} \circ X^{i_{d+1}}$ forms a linear transformation composed with a constant addition over $\mathbb{F}_2$ acting on S-box outputs. From Eq. (5), the S-box implemented by $\text{Cir}_{n_t, n_c, n_x} = C^{i_0} \circ T \circ C^{i_1} \circ T \circ \dots \circ C^{i_{d-1}} \circ T \circ C^{i_d} \circ X^{i_{d+1}}$ is affine equivalent to the one implemented by $\text{Cir}'_{n_t, n_c, n_x} = T \circ C^{i_1} \circ T \circ \dots \circ C^{i_{d-1}} \circ T$. $\square$

From the proof of Lemma 1, based on affine equivalence, we can reduce the number of candidate circuits by moving the gates between two Toffoli layers to the end of the last Toffoli gate. However, it is not appropriate to do so at all times. As shown in Figure 7 in [26], gate exchange can introduce additional gates: swapping a CNOT gate and a Toffoli gate may add Toffoli gates, swapping a Pauli-X gate and a Toffoli gate may add CNOT gates, swapping a Pauli-X gate and a CNOT gate may add CNOT gates.

For $K \leq 3$, to avoid introducing unexpected Toffoli gates and CNOT gates between two Toffoli layers, we restrict consideration to moving Pauli-X gates applied before the final Toffoli gate. In this case, $\text{Cir}_{2,0,1} = \{X \circ T^2, T \circ X \circ T, T^2 \circ X\}$ is affine equivalent to $\text{Cir}_{2,0,0}$. All the possible circuits when $K \leq 3$ can be summarized as follows:

$$\text{Cir}_{2,0,0} = \{T^2\}, \text{Cir}_{2,1,0} = \{T \circ C \circ T\}, \text{Cir}_{3,0,0} = \{T^3\}. \quad (\text{E1})$$

Similarly, for $K = 4$, possible circuits in $\text{Cir}_{2,2,0} = \{C^2 \circ T^2, T \circ C^2 \circ T, T^2 \circ C^2, C \circ T \circ C \circ T, T \circ C \circ T \circ C, C \circ T^2 \circ C\}$ are affine equivalent to $\{T^2, T \circ C^2 \circ T, T \circ C \circ T\}$, only $T \circ C^2 \circ T \in \text{Cir}_{2,2,0}$ and the rest circuits have been verified when $K \leq 3$. All the possible circuits when $K = 4$ can be summarized as follows:

$$\begin{aligned} \text{Cir}_{2,2,0} &= \{T \circ C^2 \circ T\}, \\ \text{Cir}_{2,1,1} &= \{T \circ X \circ C \circ T\}, \text{Cir}_{3,0,1} = \{T \circ X \circ T^2\}, \\ \text{Cir}_{3,1,0} &= \{T \circ C \circ T^2, T^2 \circ C \circ T\}, \text{Cir}_{4,0,0} = \{T^4\}. \end{aligned} \quad (\text{E2})$$

References

- [1] Biham, E., Shamir, A.: Differential Cryptanalysis of the Data Encryption Standard. Springer, New York (1993) <https://doi.org/10.1007/978-1-4613-9314-6>
- [2] Matsui, M.: Linear Cryptanalysis Method for DES Cipher. In: Helleseht, T. (eds) Advances in Cryptology - EUROCRYPT '93 - Workshop

- on the Theory and Application of Cryptographic Techniques Lofthus, Norway, May 23–27, 1993, Proceedings, vol. 765 of Lecture Notes in Computer Science, pp. 386–397. Springer, (1994) https://doi.org/10.1007/3-540-48285-7_33
- [3] Grover, L.K.: A fast quantum mechanical algorithm for database search. In: Gary L. Miller, (ed), In: Proceedings of the Twenty-Eighth Annual ACM Symposium on the Theory of Computing, Philadelphia, Pennsylvania, USA, May 22-24, 1996, pp. 212–219. ACM, (1996) <https://doi.org/10.1145/237814.237866>
- [4] Simon, D.R.: On the power of quantum computation. SIAM J. Comput. **26**(5), 1474–1483 (1997) <https://doi.org/10.1137/S0097539796298637>
- [5] Shor, P.W.: Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer. SIAM J. Comput. **26**(5), 1484–1509 (1997) <https://doi.org/10.1137/S0097539795293172>
- [6] Grassl, M., Langenberg, B., Roetteler, M., Steinwandt, R.: Applying grover’s algorithm to AES: quantum resource estimates. In: Takagi, T., (eds.), Post-Quantum Cryptography - PQCrypto 2016 - 7th International Workshop, Fukuoka, Japan, February 24-26, 2016, Proceedings, vol. 9606 of Lecture Notes in Computer Science, pp. 29–43. Springer, (2016) https://doi.org/10.1007/978-3-319-29360-8_3
- [7] Jaques, S., Naehrig, M., Roetteler, M., Virdia, F.: Implementing grover oracles for quantum key search on AES and lowmc. In: Canteaut, A., Ishai, Y., (eds.), Advances in Cryptology - EUROCRYPT 2020 - 39th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Zagreb, Croatia, May 10-14, 2020, Proceedings, Part II, vol. 12106 of Lecture Notes in Computer Science, pp. 280–310. Springer, (2020) https://doi.org/10.1007/978-3-030-45724-2_10
- [8] Zou, J., Wei, Z., Sun, S., Liu, X., Wu, W.: Quantum circuit implementations of aes with fewer qubits. In: Moriai, S., Wang, H. (eds.), Advances in Cryptology - ASIACRYPT 2020 - 26th International Conference on the Theory and Application of Cryptology and Information Security, Daejeon, South Korea, December 7-11, 2020, Proceedings, Part II, vol. 12492 of Lecture Notes in Computer Science, pp. 697–726. Springer, (2020) https://doi.org/10.1007/978-3-030-64834-3_24
- [9] Huang, Z., Sun, S.: Synthesizing quantum circuits of AES with lower t-depth and less qubits. In: Agrawal, S., Lin, D. (eds.), Advances in Cryptology - ASIACRYPT 2022 - 28th International Conference on the Theory and Application of Cryptology and Information Security, Taipei, Taiwan, December 5-9, 2022, Proceedings, Part III, vol. 13793 of Lecture Notes in Computer Science, pp. 614–644. Springer, (2022) https://doi.org/10.1007/978-3-030-64834-3_24

[//doi.org/10.1007/978-3-031-22969-5_21](https://doi.org/10.1007/978-3-031-22969-5_21)

- [10] NIST: Submission Requirements and Evaluation Criteria for the Post-Quantum Cryptography Standardization Process. <https://csrc.nist.gov/groups/ST/post-quantum-crypto/documents/call-for-proposals-final-dec-2016.pdf> (2016)
- [11] NIST: Call for Additional Digital Signature Schemes for the Post-Quantum Cryptography Standardization Process. <https://csrc.nist.gov/csrc/media/Projects/pqc-dig-sig/documents/call-for-proposals-dig-sig-sept-2022.pdf> (2023)
- [12] Shannon, C.E.: Communication theory of secrecy systems. *The Bell System Technical Journal* **28**(4), 656–715 (1949) <https://doi.org/10.1002/j.1538-7305.1949.tb00928.x>
- [13] Jean, J., Peyrin, T., Sim, S.M., Tourteaux, J.: Optimizing Implementations of Lightweight Building Blocks. *IACR Transactions on Symmetric Cryptology* **2017**(4), 130–168 (2017) <https://doi.org/10.13154/tosc.v2017.i4.130-168>
- [14] Bao, Z., Guo, J., Ling, S., Sasaki, Y.: PEIGEN - a Platform for Evaluation, Implementation, and Generation of S-boxes. *IACR Transactions on Symmetric Cryptology* **2019**(1), 330–394 (2019) <https://doi.org/10.13154/tosc.v2019.i1.330-394>
- [15] Li, Z., Gao, F., Qin, S., Wen, Q.: New record in the number of qubits for a quantum implementation of AES. *Frontiers in Physics* **2023**(11), 1171753 (2023) <https://doi.org/10.3389/fphy.2023.1171753>
- [16] Lin, D., Xiang, Z., Xu, R., Zhang, S., Zeng, X.: Optimized Quantum Implementation of AES. *Quantum Information Processing* **22**, 352 (2023) <https://doi.org/10.1007/s11128-023-04043-9>
- [17] Lin, D., Xiang, Z., Xu, R., Zeng, X., Zhang, S.: Quantum circuit implementations of SM4 block cipher based on different gate sets. *Quantum Information Processing* **22**, 282 (2023) <https://doi.org/10.1007/s11128-023-04002-4>
- [18] Dasu, V.A., Bakshi, A., SarKar, S., Chattopadhyay, A.: LIGHTER-R: Optimized Reversible Circuit Implementation For SBoxes. In *SOCC 2019 - 32nd IEEE International System-on-Chip Conference*, Singapore, 03-06 September 2019, pp. 260–265. IEEE, (2019) <https://doi.org/10.1109/SOCC46988.2019.1570548320>
- [19] Chun, M., Bakshi, A., Chattopadhyay, A.: DORCIS: Depth Optimized Quantum Implementation of Substitution Boxes. *IACR Cryptology*

ePrint Archive **2023**, 286 (2023) <https://eprint.iacr.org/2023/286>

- [20] Stoffelen, K.: Optimizing S-Box Implementations for Several Criteria Using SAT Solvers. In: Peyrin, T. (eds), Fast Software Encryption - FSE 2016 - 23rd International Conference, Bochum, Germany, March 20-23, 2016, Revised Selected Papers, vol. 9783 of Lecture Notes in Computer Science, pp. 140–160. Springer, (2016) https://doi.org/10.1007/978-3-662-52993-5_8
- [21] Bilgin, B., De Meyer, L., Duval, S., Levi, I., Standaert, F.X.: Low AND Depth and Efficient Inverses: a Guide on S-boxes for Low-latency Masking. IACR Transactions on Symmetric Cryptology **2020**(1), 144–184 (2020) <https://doi.org/10.13154/tosc.v2020.i1.144-184>
- [22] Fan, Y., Wang, W., Li, Z., Lu, Z., Yiu, S.M., Wang, M.: Forced Independent Optimized Implementation of 4-Bit S-Box. In: Baek, J., Ruj, S. (eds), Information Security and Privacy - ACISP 2021 - 26th Australasian Conference, Virtual Event, December 1–3, 2021, Proceedings, vol. 13083 of Lecture Notes in Computer Science, pp. 151–170. Springer, (2021) https://doi.org/10.1007/978-3-030-90567-5_8
- [23] Feng, J., Wei, Y., Zhang, F., Pasalic, E., Zhou, Y.: Novel Optimized Implementations of Lightweight Cryptographic S-Boxes via SAT Solvers. IEEE Transactions on Circuits and Systems I: Regular Papers **71**(1), 334–347 (2024) <https://doi.org/10.1109/TCSI.2023.3325559>
- [24] Lu, Z., Mesnager, S., Cui, T., Fan, Y., Wang, M.: An STP-based model toward designing S-boxes with good cryptographic properties. Designs, Codes and Cryptography **90**, 1179–1202 (2022) <https://doi.org/10.1007/s10623-022-01034-2>
- [25] Park, S., Sung, S.H., Lee, S., Lim, J.: Improving the Upper Bound on the Maximum Differential and the Maximum Linear Hull Probability for SPN Structures and AES. In: Johansson, T. (eds), Fast Software Encryption - FSE 2003 - 10th International Workshop, LUND, Sweden, February 24-26, 2003, Revised Papers, vol. 2887 of Lecture Notes in Computer Science, pp. 247–260. Springer, (2003) https://doi.org/10.1007/978-3-540-39887-5_19
- [26] Shende, V.V., Prasad, A.K., Markov, I.L., Hayes, J.P.: Synthesis of reversible logic circuits. IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems **22**(6), 710–722 (2003) <https://doi.org/10.1109/TCAD.2003.811448>
- [27] Eén, N., Sörensson, N.: An Extensible SAT-solver. In: Giunchiglia, E., Tacchella, A. (eds), Theory and Applications of Satisfiability Testing - SAT 2003 - 6th International Conference, Santa Margherita Ligure,

- Italy, May 5-8, 2003, Selected Revised Papers, vol. 2919 of Lecture Notes in Computer Science, pp. 502–518. Springer, (2004) https://doi.org/10.1007/978-3-540-24605-3_37
- [28] Soos, M., Nohl, K., Castelluccia, C.: Extending SAT Solvers to Cryptographic Problems. In: Kullmann, O. (eds), Theory and Applications of Satisfiability Testing - SAT 2009 - 12th International Conference, Swansea, UK, June 30 - July 3, 2009, Proceedings, vol. of 5584 of Lecture Notes in Computer Science, pp. 244–257. Springer, (2009) https://doi.org/10.1007/978-3-642-02777-2_24
- [29] Biere, A.: CADICAL at the SAT Race 2019. In: Heule Marijn, J.H., Matti, J., Martin, S. (eds), Proceedings of SAT RACE 2019: Solver and Benchmark Descriptions, B-2019-1 of Department of Computer Science Series of Publications, pp. 14–15. University of Helsinki, (2019)
- [30] Ganesh, V., Dill, D.L.: A Decision Procedure for Bit-Vectors and Arrays. In: Damm, W., Hermanns, H. (eds), Computer Aided Verification - CAV 2007 - 19th International Conference, Berlin, Germany, July 3-7, 2007, Proceedings, vol. 4590 of Lecture Notes in Computer Science, pp. 519–531. Springer, (2007) https://doi.org/10.1007/978-3-540-73368-3_52
- [31] Nyberg, K.: Differentially uniform mappings for cryptography. In: Helleseeth, T. (eds), Advances in Cryptology - EUROCRYPT '93 - Workshop on the Theory and Application of Cryptographic Techniques Lofthus, Norway, May 23–27, 1993, Proceedings, vol. 765 of Lecture Notes in Computer Science, pp. 55–64. Springer, (1994) https://doi.org/10.1007/3-540-48285-7_6
- [32] Dobbertin, H.: Construction of bent functions and balanced Boolean functions with high nonlinearity. In: Preneel, B. (eds), Fast Software Encryption - FSE 1994 - Second International Workshop, Leuven, Belgium, December 14-16, 1994, Proceedings, vol. 1008 of Lecture Notes in Computer Science, pp. 61–74. Springer, (1995) https://doi.org/10.1007/3-540-60590-8_5
- [33] Saeedi, M., Markov, I.L.: Synthesis and optimization of reversible circuits - a survey. *ACM Computing Surveys* **45**(2), 21 (2013) <https://doi.org/10.1145/2431211.2431220>
- [34] Leander, G., Poschmann, A.: On the Classification of 4 Bit S-Boxes. In: Carlet, C., Sunar, B. (eds), Arithmetic of Finite Fields - WAIFI 2007 - First International Workshop, WAIFI 2007, Madrid, Spain, June 21-22, 2007, Proceedings, vol. 4547 of Lecture Notes in Computer Science, pp. 159–176. Springer, (2007) https://doi.org/10.1007/978-3-540-73074-3_13

- [35] Kavun E.B., Lauridsen, M.M., Leander, G., Rechberger, C., Schwabe, P., Yalçın, T.: Prøst v1.1. CAESAR submission, <http://competitions.cr.yp.to/round1/proestv11.pdf> (2014)
- [36] Shibutani, K., Isobe, T., Hiwatari, H., Mitsuda, A., Akishita, T., Shirai, T.: Piccolo: An Ultra-Lightweight Blockcipher. In: Preneel, B., Takagi, T. (eds), Cryptographic Hardware and Embedded Systems - CHES 2011 - 13th International Workshop, Nara, Japan, September 28 - October 1, 2011, Proceedings, vol. 6917 of Lecture Notes in Computer Science, pp. 342–357. Springer, (2011) https://doi.org/10.1007/978-3-642-23951-9_23
- [37] Beierle, C., Jean, J., Kölbl, S., Leander, G., Moradi, A., Peyrin, T., Sasaki, Y., Sasdrich, P., Sim, S.M.: The SKINNY Family of Block Ciphers and Its Low-Latency Variant MANTIS . In: Robshaw, M., Katz, J. (eds), Advances in Cryptology - CRYPTO 2016 - 36th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 14-18, 2016, Proceedings, Part II, vol. 9815 of Lecture Notes in Computer Science, pp. 123–153. Springer, (2016) https://doi.org/10.1007/978-3-662-53008-5_5
- [38] Bogdanov, A., Knudsen, L.R., Leander, G., Paar, C., Poschmann, A., Robshaw, M.J.B., Seurin, Y., Vikkelsoe, C.: PRESENT: An Ultra-Lightweight Block Cipher. In: Paillier, P., Verbauwhede, I. (eds), Cryptographic Hardware and Embedded Systems - CHES 2007 - 9th International Workshop, Vienna, Austria, September 10-13, 2007, Proceedings, vol. 4727 of Lecture Notes in Computer Science, pp. 450–466. Springer, (2007) https://doi.org/10.1007/978-3-540-74735-2_31
- [39] Zhang, W., Bao, Z., Lin, D., Rijmen, V., Yang, B., Verbauwhede, I.: RECTANGLE: a bit-slice lightweight block cipher suitable for multiple platforms. *Science China Information Sciences* **58**, 1–15 (2015) <https://doi.org/10.1007/s11432-015-5459-7>
- [40] Zhang, L., Wu, W., Wang, Y., Wu, S., Zhang, J.: LAC: A lightweight authenticated encryption cipher. CAESAR submission, <http://competitions.cr.yp.to/round1/lacv1.pdf> (2014)
- [41] Banik, S., Pandey, S.K., Peyrin, T., Sasaki, Y., Sim, S.M., Todo, Y.: GIFT: A Small Present. In: Fischer, W., Homma, N. (eds), Cryptographic Hardware and Embedded Systems - CHES 2017 - 19th International Conference, Taipei, Taiwan, September 25-28, 2017, Proceedings, vol. 10529 of Lecture Notes in Computer Science, pp. 321–345. Springer, (2017) https://doi.org/10.1007/978-3-319-66787-4_16
- [42] Dobraunig, C., Eichlseder, M., Mendel, F., Schläpfer, M.: Ascon v1.1. CAESAR submission, <http://competitions.cr.yp.to/round2/asconv11.pdf> (2015)

- [43] Bertoni, G., Daemen, J., Peeters, M., Assche, G.: The Keccak reference.
<http://keccak.noekeon.org/> (2011)